BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption

Sangpyo Kim
vnb987@snu.ac.kr
Seoul National University
Seoul, South Korea

Jongmin Kim
jongmin.kim@snu.ac.kr
Seoul National University
Seoul, South Korea

Michael Jaemin Kim
michael604@snu.ac.kr
Seoul National University
Seoul, South Korea

Wonkyung Jung
wk@cryptolab.co.kr
Crypto Lab. Inc
Seoul, South Korea

John Kim
jjk12@kaist.edu
KAIST
Daejeon, South Korea

Minsoo Rhu
minsoo.rhu@gmail.com
KAIST
Daejeon, South Korea

Jung Ho Ahn
gajh@snu.ac.kr
Seoul National University
Seoul, South Korea

ABSTRACT

Homomorphic encryption (HE) enables the secure offloading of computations to the cloud by providing computation on encrypted data (ciphertexts). HE is based on noisy encryption schemes in which noise accumulates as more computations are applied to the data. The limited number of operations applicable to the data prevents practical applications from exploiting HE. Bootstrapping enables an unlimited number of operations or *fully* HE (FHE) by refreshing the ciphertext. Unfortunately, bootstrapping requires a significant amount of additional computation and memory bandwidth as well. Prior works have proposed hardware accelerators for computation primitives of FHE. However, to the best of our knowledge, this is the first to propose a hardware FHE accelerator that supports bootstrapping as a first-class citizen.

In particular, we propose BTS —Bootstrappable, Technology-driven, Secure accelerator architecture for FHE. We identify the challenges of supporting bootstrapping in the accelerator and analyze the off-chip memory bandwidth and computation required. In particular, given the limitations of modern memory technology, we identify the HE parameter sets that are efficient for FHE acceleration. Based on the insights gained from our analysis, we propose BTS, which effectively exploits the parallelism innate in HE operations by arranging a massive number of processing elements in a grid. We present the design and microarchitecture of BTS, including a network-on-chip design that exploits a deterministic communication pattern. BTS shows 5,556× and 1,306× improved execution time on ResNet-20 and logistic regression over a CPU, with a chip area of 373.6mm^2 and up to 163.2W of power.

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**;
• **Security and privacy**;

KEYWORDS

Fully Homomorphic Encryption, CKKS, Bootstrapping, Accelerator, Technology-driven

ISCA '22, June 18–22, 2022, New York, NY, USA

ACM ISBN 978-1-4503-8610-4/22/06.
https://doi.org/10.1145/3470496.3527415

1 INTRODUCTION

Homomorphic encryption (HE) allows computations on encrypted data or ciphertexts (**ct**s). In the machine-learning-as-a-service (MLaaS) era, HE is highlighted as an enabler for privacy-preserving cloud computing, as it allows safe offloading of private data. Because HE schemes are based on the learning-with-errors (LWE) [72] problem, they are noisy in nature. Noise accumulates as we apply a sequence of computations on **ct**s. This limits the number of computations that can be performed and hinders the applicability of HE for practical purposes, such as in deep-learning models with high accuracy [59]. To overcome this limitation, *fully HE* (FHE) [37] was proposed, featuring an operation (op) called *bootstrapping*, that "refreshes" the **ct** and hence permits an unlimited number of computations on the **ct**. Among multiple HE schemes that support FHE, CKKS [22] is one of the prime candidates as it supports fixed-point real number arithmetic.

One of the main barriers to adopting HE has been its high computational and memory overhead. New schemes [13, 14, 22, 24, 36] and algorithmic optimizations [3, 12, 40] (using the residue number system [7, 19]) have reduced this overhead and resulted in a 1,000,000× speedup [12] at least compared to its first HE implementation [38]. However, even with such efforts, HE ops experience tens of thousands of slowdowns compared to unencrypted ops [49]. Attempting to tackle this, prior works have sought hardware solutions to accelerate HE ops, including CPU extensions [11, 49], GPU [2–4, 48], FPGA [53, 54, 73, 74], and ASIC [75].

However, prior acceleration works mostly targeted small problem sizes, with a small target N (the length of a **ct**), and they are lacking in bootstrapping support. Bootstrapping, which is necessary to reduce the impact of noise, occurs frequently in most FHE applications and represents the highest expense. For example, bootstrapping occurs more than 1,000 times for a single ResNet-20 inference [59] and each instance of bootstrapping can take dozens of seconds on the state-of-the-art CPU [35] and hundreds of milliseconds on a GPU [48]. Most prior custom hardware acceleration works [73, 74] do not support bootstrapping at all, while `F1` [75] demonstrated a bootstrapping time for CKKS but with limited throughput (Table 1).

Table 1: Comparing prior HE acceleration works with BTS

	Plat -form	Target **ct** len (N)	Boot -strap -pable	Refreshed slots† per bootstrap	Data parallel -ism‡	FHE mult thruput ($1/s$)
Lattigo [35]	CPU	2^{16}	○	32,768	-	6-10K
100x [48]	GPU	2^{17}	○	65,536	SIMT	0.1-1M
[74]	FPGA	2^{12}	×	-	rPLP	×
HEAX [73]	FPGA	2^{14}	×	-	rPLP	×
F1 [75]	ASIC	2^{14}	△*	1	rPLP	4K
BTS	ASIC	2^{17}	○	65,536	CLP	20M

† Data elements that can be packed in a **ct** for SIMD execution.
‡ Residue-polynomial-level parallelism (rPLP) and coefficient-level parallelism (CLP) can be exploited in parallelizing HE ops (Section 4.3).
* F1 only supports single-slot bootstrapping which has low throughput.

We propose BTS, a bootstrapping-oriented FHE accelerator that is **B**ootstrappable, **T**echnology-driven, and **S**ecure. First, we identify the limitations that are imposed by contemporary fabrication technology when designing an HE accelerator, analyzing the implications of various conflicting requirements for the performance and security of FHE under such a constrained design space. This allows us to pinpoint appropriate optimization targets and requirements when designing the FHE accelerator. Second, we build a balanced architecture on top of those observations; we analyze the characteristics of HE functions to determine the appropriate number of processing elements (PEs) and proper data mapping that balances computation and data movement when using our FHE-optimized parameters. We also choose to exploit coefficient-level parallelism (CLP), instead of residue-polynomial-level parallelism (rPLP), to evade the load imbalance issue. Finally, we devise a novel PE microarchitecture that efficiently handles HE functions including base conversion, and a time-multiplexed NoC structure that manages both number theoretic transform and automorphism functions.

Through these detailed studies, BTS achieves a 5,714× speedup in multiplicative throughput against F1, the state-of-the-art ASIC implementation, when bootstrapping is properly considered. Also, BTS significantly reduces the training time of logistic regression [39] compared to the CPU (by 1,306×) and GPU (by 27×) implementations, and can execute a ResNet-20 inference 5,556× faster than the prior CPU implementation [59].

In this paper, we make the following key contributions:

- We provide a detailed analysis of the interplay of HE parameters impacting the performance of FHE accelerators.
- We propose BTS, a novel accelerator architecture equipped with massively parallel compute units and NoCs tailored to the mathematical traits of FHE ops.
- BTS is the first accelerator targeting practical bootstrapping, enabling unbounded multiplicative depth, which is essential for complex workloads.

2 BACKGROUND

We provide a brief overview of HE and CKKS [22] in particular. Table 2 summarizes the key parameters and notations we use in this paper.

Table 2: List of symbols used to describe CKKS [22].

Symbol	Definition
Q	(Prime) moduli product = $\prod_{i=0}^{L} q_i$
$q_0, ..., q_L$	(Prime) moduli
$Q_0, ..., Q_{\mathsf{dnum}-1}$	Modulus factors
P	Special (prime) moduli product = $\prod_{i=0}^{k-1} p_i$
$p_0, ..., p_{k-1}$	Special (prime) moduli
$\mathbf{evk}_{mult}$	Evaluation key (**evk**) for HMult
$\mathbf{evk}_{rot}^{(r)}$	**evk** for HRot with a rotation amount of r
N	The degree of a polynomial
L	Maximum (multiplicative) level
ℓ	Current (multiplicative) level of a ciphertext
L_{boot}	Levels consumed at bootstrapping
k	The number of special prime moduli
dnum	Decomposition number
λ	Security parameter of a given CKKS instance

2.1 Homomorphic Encryption (HE)

HE enables direct computation on encrypted data, referred to as ciphertext (**ct**), without decryption. There are two types of HE. *Leveled HE (LHE)* supports a limited number of operations (ops) on a **ct** due to the noise that accumulates after the ops. In contrast, *Fully HE (FHE)* allows an unlimited number of ops on **ct**s through bootstrapping [37] that "refreshes" a **ct** and lowers the impact of noise. LHE has limited applicability[1]; in the field of privacy-preserving deep learning inference, for instance, simple/shallow networks such as LoLa [15] can be implemented with LHE, but only with limited accuracy (74.1%). More accurate models such as ResNet-20 [59] (92.43%) demand much more ops applied to **ct**s and thus FHE implementation.

While other FHE schemes support integer [13, 14, 36] or boolean [24] data types, CKKS [22] supports fixed-point complex (real) numbers. As many real-world applications such as MLaaS (Machine Learning as a Service) require arithmetic on real numbers, CKKS has become one of the most prominent FHE schemes. In this paper, we focus on accelerating CKKS ops; however, our proposed architecture is applicable to other popular FHE schemes (e.g., BGV [13] and BFV [7, 14, 36]) that share similar core ops.

2.2 CKKS: an emerging HE scheme

CKKS first encodes a message that is a vector of complex numbers, into a plaintext $m(X) = \sum_{i=0}^{N-1} c_i X^i$, which is a polynomial in a cyclotomic polynomial ring $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N+1)$. The coefficients $\{c_i\}$ are integers modulo Q and the number of coefficients (or degree) is N, where N is a power-of-two integer, typically ranging from 2^{10} to 2^{18}. For a given N, a message with up to $N/2$ complex numbers can be *packed* into a single plaintext in CKKS. Each element within a packed message is referred to as a *slot*. After encoding

[1]The hybrid use of LHE with multi-party computation [31] allows for a broader range of applications. However, such an approach has a different bottleneck of the communication cost and intense client-side computations.

(or packing), element-wise multiplication (mult) and addition between two messages can be done through polynomial operations between plaintexts. CKKS then encrypts a plaintext $m(X) \in \mathcal{R}_Q$ into a $\mathbf{ct} \in \mathcal{R}_Q^2$ based on the following equation,

$$\mathbf{ct} = (b(X), a(X)) = (a(X) \cdot s(X) + m(X) + e(X), a(X))$$

where $s(X) \in \mathcal{R}_Q$ is a secret key, $a(X) \in \mathcal{R}_Q$ is a random polynomial, and $e(X)$ is a small Gaussian error polynomial required for LWE security guarantee [5]. CKKS decrypts $\mathbf{ct}$ by computing $m'(X) = \mathbf{ct} \cdot (1, -s(X)) = m(X) + e(X)$, which approximates to $m(X)$ with small errors.

HE is mainly bottlenecked by the high computational complexity of polynomial ops. As each coefficient of a polynomial is a large integer (up to 1,000s of bits) and the degree is high (even surpassing 100,000), an op between two polynomials has high compute and data-transfer costs. To reduce the computational complexity, HE schemes using the residue number system (RNS) [7, 19] have been proposed. For example, Full-RNS CKKS [19] sets Q as the product of word-sized *(prime) moduli* $\{q_i\}_{0 \le i \le L}$, where $Q = \prod_{i=0}^{L} q_i$ for a given integer L. Using the Chinese remainder theorem (Eq. 1), we represent a polynomial in $\mathcal{R}_Q$ with *residue polynomials* in $\{\mathcal{R}_{q_i}\}_{0 \le i \le L}$, whose coefficients are residues obtained by performing modulo q_i (represented as $[\cdot]_{q_i}$) on the large coefficients:

$$[a(X)]_Q \mapsto ([a(X)]_{q_0}, \ldots, [a(X)]_{q_L}) \text{ where } Q = \prod_i q_i \quad (1)$$

Then, we can convert an op involving two polynomials into ops between the residue polynomials with word-sized coefficients ($\le$ 64 bits) corresponding to the same q_i, avoiding costly big-integer arithmetic with carry propagation. Full-RNS CKKS provides an ~8× speedup over plain CKKS [19] and thus, we adopt Full-RNS CKKS as our CKKS implementation, representing a polynomial in $\mathcal{R}_Q$ as an $N \times (L+1)$ matrix of residues, and a $\mathbf{ct}$ as a pair of such matrices.

2.3 Primitive operations (ops) of CKKS

Primitive HE ops of CKKS are introduced here, which can be combined to create more complex HE ops such as linear transformation and convolution. Given two ciphertexts $\mathbf{ct}_0, \mathbf{ct}_1$ where $\mathbf{ct}_i = (b_i(X), a_i(X))$ and $b_i(X) = a_i(X) \cdot s(X) + m_i(X)$, HE ops can be summarized as follows:

HAdd performs an element-wise addition of $\mathbf{ct}_0$ and $\mathbf{ct}_1$:

$$\mathbf{ct}_{add} = (b_0(X) + b_1(X), a_0(X) + a_1(X)) \quad (2)$$

HMult consists of a *tensor product* and *key-switching*. The tensor product first creates $(d_0(X), d_1(X), d_2(X))$:

$$\begin{aligned} d_0(X) &= b_0(X) \cdot b_1(X) \\ d_1(X) &= a_0(X) \cdot b_1(X) + a_1(X) \cdot b_0(X) \\ d_2(X) &= a_0(X) \cdot a_1(X) \end{aligned} \quad (3)$$

By computing $(d_0(X), d_1(X), d_2(X)) \cdot (1, -s(X), s(X)^2)$, we recover $m_0(X) \cdot m_1(X)$, albeit with error terms. Key-switching recombines the tensor product result to be decryptable with $(1, -s(X))$ using a public key, called an evaluation key ($\mathbf{evk}$). An $\mathbf{evk}$ is a $\mathbf{ct}$ in $\mathcal{R}_{PQ}^2$ with a larger modulus PQ, where $P = (\prod_{i=0}^{k-1} p_i) \ge Q$ for given *special (prime) moduli* $p_0, \ldots, p_{k-1}$. We express an $\mathbf{evk}$ as a pair of $N \times (k+L+1)$ matrices. HMult is then computed using Eq. 4, which involves key-switching with an $\mathbf{evk}$ for mult, $\mathbf{evk}_{mult}$:

$$\mathbf{ct}_{mult} = (d_0(X), d_1(X)) + \underbrace{P^{-1}(d_2(X) \cdot \mathbf{evk}_{mult})}_{\text{key-switching}} \quad (4)$$

HRot circularly shifts a message vector by slots. When a $\mathbf{ct}$ encrypts a message vector $\mathbf{z} = (z_0, ..., z_{N/2-1})$, after applying HRot by a *rotation amount* r, the rotated ciphertext $\mathbf{ct}_{rot}$ encrypts $\mathbf{z}^{(r)} = (z_r, ..., z_{N/2-1}, z_0, ..., z_{r-1})$. HRot consists of an *automorphism* and key-switching. $\mathbf{ct} = (b(X), a(X))$ is mapped to $\mathbf{ct'} = (b(X^{5^r}), a(X^{5^r}))$ after an automorphism. This moves the coefficients of a polynomial through the mapping $i \mapsto \sigma_r(i)$, where i is the index of the coefficient c_i and σ_r is:

$$\sigma_r : i \mapsto i \cdot 5^r \bmod N \quad (i = 0, 1, \ldots, N-1) \quad (5)$$

Similar to HMult, key-switching brings back $\mathbf{ct'}$, which was only decryptable with $(1, -s(X^{5^r}))$ after automorphism, to be decryptable with $(1, -s(X))$. An HRot with a different rotation amount each requires a separate $\mathbf{evk}$, $\mathbf{evk}_{rot}^{(r)}$. HRot is computed as follows:

$$\mathbf{ct}_{rot} = (b(X^{5^r}), 0) + P^{-1}(a(X^{5^r}) \cdot \mathbf{evk}_{rot}^{(r)}) \quad (6)$$

HE applications require other HE ops, such as an addition or mult of a $\mathbf{ct}$ with a scalar (**CAdd**, **CMult**) or a polynomial (**PAdd**, **PMult**) of unencrypted, constant values. Additions are performed by adding the scalar or polynomial to $b(X)$, and mults are performed by multiplying each $b(X)$ and $a(X)$ by the scalar or polynomial.

2.4 Multiplicative level and HE bootstrapping

Multiplicative level: The error included in a $\mathbf{ct}$ is amplified during HE ops; in particular, HMult multiplies the error $e(X)$ with other terms (e.g., $m_0(X)$ and $m_1(X)$) and can result in an explosion of the error if not treated properly. CKKS performs **HRescale** to mitigate this explosion and keep the error tolerable by dividing the $\mathbf{ct}$ by the last prime modulus q_L [19]. After HRescale, the q_L residue polynomial is discarded, and the $\mathbf{ct}$ is reduced in size. The $\mathbf{ct}$ continues losing the residues of $q_{L-1}, \ldots, q_1$ with each HRescale while executing an HE application until only one residue polynomial is left when no additional HMult can be performed on the $\mathbf{ct}$. L, or the *maximum multiplicative level*, determines the maximum number of HMult ops that can be performed without bootstrapping, and the current *(multiplicative) level* ℓ denotes the number of remaining HMult operations that can be performed on the $\mathbf{ct}$. Thus, a $\mathbf{ct}$ with a level ℓ is represented as a pair of $N \times (\ell+1)$ matrices.

Bootstrapping: FHE features a *bootstrapping* op that restores the multiplicative level (ℓ) of a $\mathbf{ct}$ to enable more ops. Bootstrapping must be commonly performed for the practical usage of HE with a complex sequence of HE ops. Bootstrapping mainly consists of homomorphic linear transforms and approximate sine evaluation [20], which can be broken down into hundreds of primitive HE ops. HMult and HRot ops account for more than 77% of the bootstrapping time [35]. As bootstrapping itself consumes L_{boot} levels, L should be larger than L_{boot}. A larger L is beneficial as it requires less frequent bootstrapping. L_{boot} ranges from 10 to 20 depending on the bootstrapping algorithm; a larger L_{boot} allows the use of more precise and faster bootstrapping algorithms[12, 17, 40, 58]. The bootstrapping algorithm we use in this paper is based on [40]

with updates to meet the latest security and precision requirements [12, 21, 60], and the value of L_{boot} is 19. Readers are encouraged to refer to the papers for a more detailed explanation of the algorithm. Another CKKS-specific constraint is that the moduli q_i's and the special moduli p_i's must be large enough to tolerate the error accumulated during bootstrapping, whose typical values range from 2^{40} to 2^{60} [23, 35].

2.5 Modern algorithmic optimizations in CKKS and amortized mult time per slot

Security level (λ): The level of security for an HE scheme is represented by λ, a parameter measured by the logarithmic time complexity for an attack [21] to deduce the secret key. A sufficiently high λ is required for safety; we target λ of 128 bits, adhering to the standard [5] established by recent HE studies [12, 58, 60] and libraries [35, 67]. λ is a strictly increasing function of $N/\log PQ$ [30].
Dnum: Key-switching is an expensive function, accounting for most of the time in HRot and HMult [48]. We adopt a state-of-the-art generalized key-switching technique [40], which balances L, the computational cost, and λ. [40] factorizes the moduli product Q into $Q = Q_0 \cdot \ldots \cdot Q_{\mathsf{dnum}-1}$ (see Eq. 7) for a given integer dnum (*decomposition number*). It decomposes a **ct** into dnum slices, each consisting of residue polynomials corresponding to the prime moduli (q_i's) that together compose the modulus factor Q_j. We perform key-switching on each slice in $\mathcal{R}_{Q_j}$ and later accumulate them. The special moduli product P should only satisfy $P \geq Q_j$ for each Q_j, allowing us to choose a smaller P, leading to a higher λ. i) Therefore, a larger dnum means *a greater level* of L with fixed values of λ and N because we can increase Q.

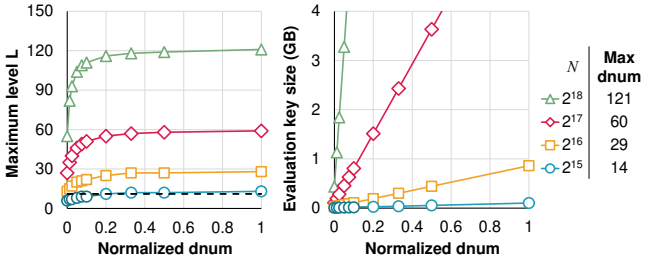
$$Q = \underbrace{q_0 \cdot \ldots \cdot q_{\frac{L+1}{\mathsf{dnum}}-1}}_{Q_0} \cdot \underbrace{q_{\frac{L+1}{\mathsf{dnum}}} \cdot \ldots \cdot q_{2\frac{L+1}{\mathsf{dnum}}-1}}_{Q_1} \cdot \ldots \cdot \underbrace{q_{(\mathsf{dnum}-1)\frac{L+1}{\mathsf{dnum}}} \cdot \ldots \cdot q_{L+1}}_{Q_{\mathsf{dnum}-1}} \quad (7)$$

A major challenge of generalized key-switching is that different **evk**s ($\mathbf{evk}_0, \ldots, \mathbf{evk}_{\mathsf{dnum}-1}$) must be prepared for each factor Q_j, where each **evk** is a pair of $N \times (k+L+1)$ matrices and k is set to $(L+1)/\mathsf{dnum}$. ii) Thus, the aggregate **evk** size becomes $2 \cdot N \cdot (L+1) \cdot (dnum+1)$, linearly increasing with dnum. iii) The overall computational complexity of a single HE op also increases with dnum. Therefore, choosing an appropriate dnum crucially affects the performance.
Amortized mult time per slot ($\mathrm{T}_{\mathrm{mult,a/slot}}$): Changing the HE parameter set has mixed effects on the performance of HE ops. Decreasing N reduces the computational complexity and memory usage. However, we should lower L and Q to sustain security, which requires more frequent bootstrapping. Also, because a **ct** of degree N can encode only up to $N/2$ message *slots* by packing, the throughput degrades.

Jung et al.[48] introduced a metric called *amortized mult time per slot* ($\mathrm{T}_{\mathrm{mult,a/slot}}$), which is calculated as follows:

$$\mathrm{T}_{\mathrm{mult,a/slot}} = \frac{\mathrm{T}_{\mathrm{boot}} + \sum_{\ell=1}^{L-L_{boot}} \mathrm{T}_{\mathrm{mult}}(\ell)}{L - L_{boot}} \cdot \frac{2}{N} \quad (8)$$

where $\mathrm{T}_{\mathrm{boot}}$ is the bootstrapping time and $\mathrm{T}_{\mathrm{mult}}(\ell)$ is the time required to perform HMult at a level ℓ. This metric initially calculates the average cost of mult including the overhead of bootstrapping, and then divides it by the number of slots in a **ct** ($N/2$). Thus,



(a) Maximum level L (b) **evk** size

Figure 1: (a) L and (b) a single evk size vs. dnum **for four different N (polynomial degree) values and a fixed 128b security target. Normalized-**dnum **of 0 means** dnum **= 1 and normalized-**dnum **of 1 means** dnum **= max (i.e., k = 1). Interpolated results are used for points with non-integer** dnum **values. The dotted line in (a) represents the minimum required level of 11 for bootstrapping.**

$\mathrm{T}_{\mathrm{mult,a/slot}}$ effectively captures the reciprocal throughput of a *CKKS instance* (CKKS scheme with a certain parameter set).

3 TECHNOLOGY-DRIVEN PARAMETER SELECTION OF BOOTSTRAPPABLE ACCELERATORS

3.1 Technology trends regarding memory hierarchy

Domain-specific architectures (e.g., deep-learning [47, 55, 61] and multimedia [70] accelerators) are often based on custom logic and an optimized dataflow to provide high computation capabilities. In addition, the memory capacity/bandwidth requirements of the applications are exploited in the design of the memory hierarchy. Recently, on-chip SRAM capacities have scaled significantly [6] such that the level of hundreds of MBs of on-chip SRAM is feasible, providing tens of TB/s of SRAM bandwidth[47, 55, 69]. While the bandwidth of the main-memory has also increased, its aggregate throughput is still more than an order of magnitude lower than the on-chip SRAM bandwidth [66], achieving a few TB/s of throughput even with high-bandwidth memory (HBM).

Similar to other domain-specific architectures [18, 47], HE applications also follow deterministic computational flows, and the locality of the input and output **ct**s of HE ops can be maximized through software scheduling [32]. Thus, **ct**s can be reused by exploiting a large amount of on-chip SRAM enabled by technology scaling. However, even with the increasing on-chip SRAM capacity, we observe that the size of on-chip SRAM is still insufficient to store **evk**s, rendering the off-chip memory bandwidth becomes a crucial bottleneck for modern CKKS scheme that supports bootstrapping. In the following sections, we identify the importance of bootstrapping on the overall performance and provide an analysis of how different CKKS parameters impact the amount of data movement during bootstrapping and its final throughput.

3.2 Interplay between key CKKS parameters

Selecting one parameter of a CKKS instance has a multifaceted effect on the other parameters. First, λ is lowered when Q is higher, and is

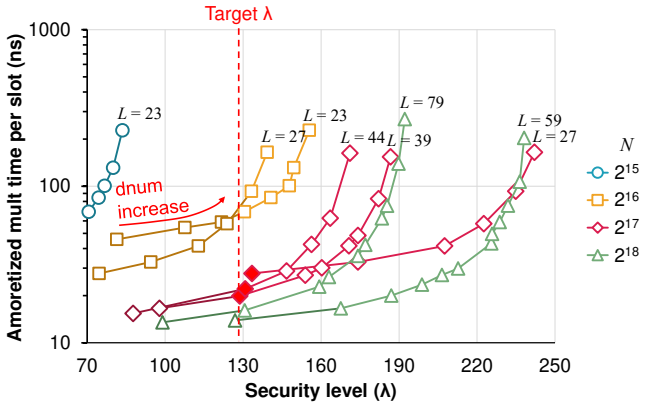


Figure 2: λ and the minimum bound $T_{mult,a/slot}$ of an HE accelerator simulated for different CKKS instances. Results are measured for all possible integer dnum values including 1 and the max for each (N, L) pair. The points highlighted in red represent (N, L, dnum) = (2^{17}, 27, 1), (2^{17}, 39, 2), (2^{17}, 44, 3).

raised when N is higher. Considering that a bootstrappable CKKS instance requires a high L ($> L_{boot}$), and with the sizes of prime moduli q_i and p_i set around 2^{50} and 2^{60} with a 64-bit machine word size, $\log PQ$ exceeds 500. To support 128b security when $\log PQ$ exceeds 500, N must be larger than 2^{14} [60].

Second, when $\log PQ$ is set from fixed values of λ and N, a larger dnum leads to a higher L at the cost of a larger **evk** size. Considering that k equals $(L+1)/\text{dnum}$, the $Q:P$ ratio is close to dnum:1. Therefore, when $\log PQ$ is fixed, a larger dnum means a larger Q and finally a larger L. However, the **evk** size also increases linearly with dnum (see Fig. 1). Because the high level of L achieved by increasing dnum saturates quickly, choosing a proper dnum is important.

3.3 Realistic minimum bound of HE accelerator execution time

$T_{mult,a/slot}$ is mainly determined by the bootstrapping time, as bootstrapping is more than 60× longer than a single HMult on conventional systems [35, 48]. Unlike simple LHE tasks such as LoLa [15], which only requires a handful of **evk**s, bootstrapping typically requires more than 40 **evk**s, mostly for the long sequence of multiple HRots applied with different r's during the linear transformation steps of bootstrapping [12] ($\mathbf{evk}_{rot}^{(r)}$). They can amount to GBs of storage and exhibit poor locality.

The bootstrapping time is mostly spent on HMult and HRot. [48] found that HMult and HRot are memory-bound, highly dependent on the on-chip storage capacity. Given today's technology with low logic costs and high-density on-chip SRAMs, the performance of HMult and HRot can be improved significantly with an HE accelerator.

Despite such an increase in on-chip storage, **evk**s, with each possibly taking up several hundreds of MBs (see Fig. 1), cannot easily be stored on-chip. Because on-chip storage cannot hold all **evk**s, they must be stored off-chip and be loaded in a streaming fashion upon every HMult/HRot. Therefore, even if every temporal data and **ct**s with high locality are assumed to be stored on-chip with massive on-chip storage, the load time of **evk** becomes the minimum execution time for HMult/HRot considering the limited off-chip bandwidth.

3.4 Desirable target CKKS parameters for HE accelerators

To understand the impact of CKKS parameters, we simulate $T_{mult,a/slot}$ at multiple points while sweeping the N, L, and dnum values. With 1TB/s of memory bandwidth (half of NVIDIA A100 [25] and identical to F1 [75]), a bootstrapping algorithm that consumes 19 levels, and the simulation methodology in Section 6.2, we add two simplifying assumptions based on Section 3.3 such that 1) the computation time of HE ops can be fully hidden by the memory latency of **evk**s, and 2) all **ct**s of HE ops are stored in on-chip SRAM and re-used. Fig. 2 reports the results. The x-axis shows λ determined by $N/\log PQ$ [30], as calculated using an estimation tool [77]. The y-axis shows $T_{mult,a/slot}$ for different Ns, Ls, and dnums.

We make two key observations. First, when other values are fixed, $T_{mult,a/slot}$ decreases as N increases, even with the higher memory pressure from the larger **ct**s and **evk**s because the available level ($L - L_{boot}$) increases. However, such an effect saturates after $N = 2^{17}$. Around our target security level of 128b in Fig. 2, the gain from 2^{16} to 2^{17} is 3.8× (111.4ns to 29.1ns), whereas that from 2^{17} to 2^{18} is 1.3×. Second, while a higher dnum can help smaller Ns to reach our target 128b security level, it comes at the cost of a superlinear increase in $T_{mult,a/slot}$ due to the increasing **evk** size and the additional gain in L being saturated.

These key observations suggest that a bootstrappable HE accelerator should target CKKS instances with *high polynomial degrees* ($N \geq 2^{17}$) and *low* dnum values. Our BTS targets the CKKS instances with $N = 2^{17}$ highlighted in Fig. 2. With these, the simulated HE accelerator achieves $T_{mult,a/slot}$ of 27.7ns, 19.9ns, and 22.1ns with corresponding (L, dnum) pairs of (27, 1), (39, 2), and (44, 3), respectively. Although BTS can support all CKKS instances shown in Fig. 2, it is not optimized for other CKKS instances as they either exhibit worse $T_{mult,a/slot}$, or require significantly more on-chip resources with only a marginal performance gain ($N = 2^{18}$).

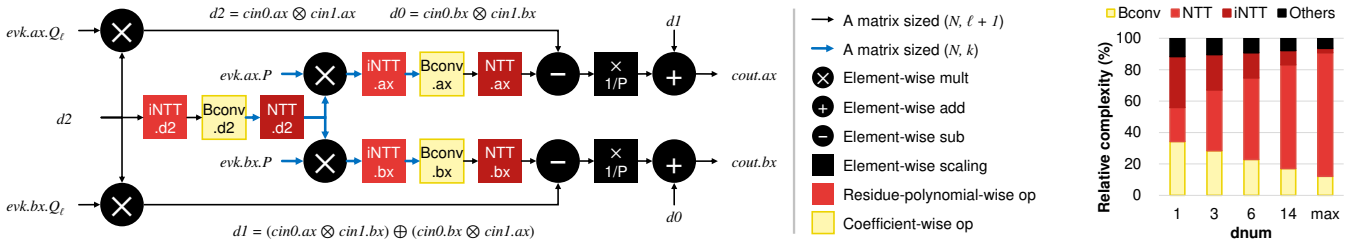
In this paper, we use the CKKS instance with $N = 2^{17}$, $L = 27$, and dnum = 1 as a running example. When using the 64-bit machine word size, a **ct** at the maximum level has a size of 56MB, and an **evk** has a size of 112MB.

4 ARCHITECTING BTS

We explore the organization of BTS, our HE accelerator architecture. We address the limitations of prior works, F1 [75] in particular, and suggest a suitable architecture for bootstrappable CKKS instances. Section 3.4 derived the optimality of such CKKS instances assuming that an HE accelerator can hide all the computation time within the loading time of an **evk**. BTS exploits massive parallelism innate in HE ops to satisfy that optimality requirement indeed, with enough, but not an excess of, functional units (FUs). To achieve this, first we dissect key-switching, which appears in both HMult and HRot, and has both heavy computation and memory requirements.

4.1 Computational breakdown of HE ops

We first dissect key-switching, which appears in both HMult and HRot, the two dominant HE ops for bootstrapping and general HE



(a) Computational flow (b) Relative complexity

Figure 3: (a) Computational flow of the key-switching inside HMult and (b) computational complexity breakdown of HMult for cts at the maximum level on CKKS instances with the same $N = 2^{17}$ and $\lambda = 128$ values but different dnum values. The computational complexity is analyzed based on [48].

workloads. Fig. 3(a) shows the computational flow of key-switching, and Fig. 3(b) shows the corresponding computational complexity breakdown. We focus on three functions, *NTT*, *iNTT*, and *BConv*, which take up most of the computation.

Number Theoretic Transform (NTT): A polynomial mult between polynomials in R_Q translates to a negacyclic convolution of their coefficients. NTT is a variant of the Discrete Fourier Transform (DFT) in R_Q. Similar to DFT, NTT transforms the convolution between two sets of coefficients into an element-wise mult, while inverse NTT (iNTT) is applied to obtain the final result as shown below ($\otimes$ meaning element-wise mult):

$$a_1(X) \cdot a_2(X) = \text{iNTT}(\text{NTT}(a_1(X)) \otimes \text{NTT}(a_2(X)))$$

By applying the well-known Fast Fourier Transform (FFT) algorithms [28], the computational complexity of (i)NTT is reduced from $O(N^2)$ to $O(N \log N)$. This strategy divides the computation into $\log N$ stages, where N data elements are paired into $N/2$ pairs in a strided manner and butterfly operations are applied to each pair per stage. The stride value changes every stage. Butterfly operations in (i)NTT are as follows:

$$\text{Butterfly}_{\text{NTT}}(X, Y, W) \rightarrow X' = X + W \cdot Y,\ Y' = X - W \cdot Y$$
$$\text{Butterfly}_{\text{iNTT}}(X, Y, W^{-1}) \rightarrow X' = X + Y,\ Y' = (X - Y) \cdot W^{-1}$$

where W (a *twiddle factor*) is an odd power (up to $2N-1$) of the primitive $2N$-th root of unity ξ. In total, N twiddle factors are needed *per prime modulus*. NTT can be applied concurrently to each residue polynomial (in R_{q_i}) in a **ct**.

Base Conversion (BConv): BConv [7] converts a set of residue polynomials to another set whose prime moduli are different from the former. A **ct** at level ℓ has two polynomials, with each consisting of $(\ell + 1)$ residue polynomials corresponding to prime moduli $\{q_0, ..., q_\ell\}$. We denote this modulus set as C_ℓ, called the polynomial's base or *base* in short.

BConv is required in key-switching to match the base of a **ct** with an **evk** on base $B \cup C_\ell$ where $B = \{p_0, ..., p_{k-1}\}$. BConv from C_ℓ to B is performed on **ct**s, as expressed in Eq. 9, where $\hat{q_j} = \prod_{i \neq j} q_i$ for $q_i \in C_\ell$. Likewise, BConv from B to C_ℓ is performed after multiplying **ct** by **evk**.

$$\underset{C_\ell \rightarrow B}{\text{BConv}}([a(X)]_{C_\ell}) = \left\{ \left[\sum_{j=0}^{\ell} \underbrace{[[a(X)]_{q_j} \cdot \hat{q_j}^{-1}]_{q_j}}_{(1)} \cdot \hat{q_j} \right]_{p_i} \right\}_{0 \leq i < k} \quad (9)$$

Because BConv cannot be performed on polynomials after NTT (i.e., they are in the *NTT domain*), iNTT is performed to bring the polynomials back to the *RNS domain*. BTS keeps polynomials in the NTT domain by default and brings them back to the RNS domain only for BConv. Thus, a sequence of iNTT → BConv → NTT is a common pattern in CKKS.

4.2 Limitations in prior works and the balanced design of BTS

Prior HE acceleration studies [71, 73–75] identified (i)NTT as the paramount acceleration target and placed multiple *NTT units* (NTTUs) that can perform both $\text{Butterfly}_{\text{NTT}}$ and $\text{Butterfly}_{\text{iNTT}}$. F1 [75] in particular populated numerous NTTUs with "the more the better" approach, provisioning 14,336 NTTUs even for a small HE parameter set with $N = 2^{14}$. Such an approach was viable because, under the small parameter sets, all **ct**, **evk**, and temporal data could reside on-chip, especially with proper compiler support.

However, we observe that such massive use of NTTUs is wasteful in bootstrappable CKKS instances, where the off-chip memory bandwidth becomes the main determinant of the overall performance. The FHE-optimized parameters cause a quadratic increase in **ct**, **evk**, and the temporal data (e.g., 64× when moving from 2^{14} to 2^{17} of N). This makes it impossible for these components to be located on-chip, especially considering that most prior custom hardware works only take into account the max dnum case.

We instead analyze how many fully-pipelined NTTUs an HE accelerator requires to finish HMult or HRot within the **evk** loading time with our target CKKS instances. We define the minimum required number of NTTUs ($\min_{\text{NTTU}}$) as $\frac{\#\text{ of butterflies per HE op}}{\text{operating frequency}} / \frac{\text{size of an evk}}{\text{main-memory bandwidth}}$. When we assume a nominal operating frequency of 1.2GHz for NTTUs considering prior works [25, 47, 55] in 7nm process nodes, and HBM with an aggregate bandwidth of 1TB/s, $\min_{\text{NTTU}}$ is defined as shown below:

$$\min_{\text{NTTU}} = \frac{(\text{dnum}+2) \cdot (k+\ell+1) \cdot \frac{1}{2} N \log N / (1.2\text{GHz})}{2 \cdot \text{dnum} \cdot (k+\ell+1) \cdot N \cdot 8\text{B} / (1\text{TB/s})} \quad (10)$$

The value of $\min_{\text{NTTU}}$ is maximized when dnum is 1. For $N = 2^{17}$, the value is 1,328. We utilize 2,048 NTTUs in BTS to provide some margin for other operations.

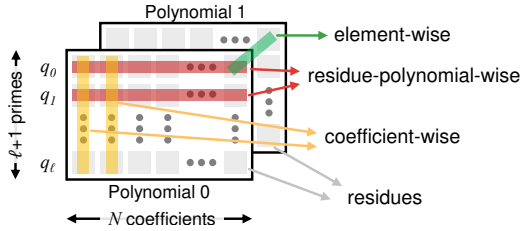


Figure 4: Data access patterns in HE functions.

In addition to (i)NTT, the importance of BConv grows as small dnums are used. The computational complexity of BConv in key-switching is proportional to $(1+\frac{2}{\text{dnum}})$. As a result, the relative computational complexity of BConv, which is 12% at dnum = max, increases to 34% at dnum = 1 (see Fig. 3(b)). Prior works mainly targeted dnum = max, focusing on the acceleration of (i)NTT. We propose a novel *BConv unit* (BConvU) to handle the increased significance of BConv, whose details are described later in Section 5.2.

4.3 BTS organization exploiting data parallelism

We can categorize primary HE functions into three groups according to their data access patterns (see Fig. 4). Residue-polynomial-wise functions, the (i)NTT and automorphism functions, involve all N residues in a residue polynomial to produce an output. Coefficient-wise functions (e.g., BConv) involve all $(\ell+1)$ residues of a single coefficient to produce an output residue. Element-wise functions such as CMult and PMult only involve residues on the same position over multiple polynomials.

We can exploit two types of data parallelism, *residue-polynomial-level parallelism* (rPLP) and *coefficient-level parallelism* (CLP), when parallelizing an HE op with multiple processing elements (PEs). rPLP can be exploited by distributing $(\ell+1)$ residue polynomials and CLP can be by distributing N coefficients to many PEs. Prior works including F1 mostly exploited rPLP as prime-wise modularization is apparently possible.

When the data access pattern and the type of the parallelism being exploited are not aligned, data exchanges between PEs occur, resulting in global wire communication which has poorly scaled over technology generations [41]. For the sequence of iNTT → BConv → NTT in key-switching, CLP will incur data exchanges for (i)NTT and rPLP will incur data exchanges for BConv. The total size of the transferred data is identical at $(k+\ell+1)N$. Thus, there is no clear winner between the two types of parallelism in terms of data exchanges. However, exploiting rPLP is limited in terms of the degree of parallelism due to the fluctuating multiplicative level ℓ as an FHE application is executed. This also complicates a fair distribution of jobs among PEs.

Instead, we use CLP in BTS. As N is fixed throughout the running of an HE application, we decide on a fixed data distribution methodology, where the residues of a polynomial with the same coefficient index are allocated to the same PE. Then, coefficient-wise and element-wise functions are parallelized without inter-PE data exchanges; only (i)NTT and the automorphism incur inter-PE data exchanges, with the communication pattern predetermined by the fixed data distribution.

We place 2,048 PEs (Eq. 10) in BTS. Each PE has an NTTU, a BConvU, a modular adder (ModAdd) and a multiplier (ModMult) for element-wise functions, as well as an SRAM scratchpad. $N=2^{17}$ residues of a residue polynomial are evenly distributed to the PEs, such that one PE handles 2^6 residues. Then six out of 17 (i)NTT stages can be solely computed inside a PE. We adopt *3D-NTT* to minimize the data exchanges between the PEs. A residue polynomial is regarded as a 3D data structure of size $2^6 \times 2^5 \times 2^6$. Then, each PE performs a sequence of 2^6-, 2^5-, and 2^6-point (i)NTTs, interleaved with just two rounds of inter-PE data exchange. Splitting (i)NTT in a more fine-grained manner requires more data exchange rounds and is thus less energy-efficient. The automorphism function exhibits a different communication pattern from (i)NTT, involving complex data remapping (Eq. 5). Nevertheless, the data distribution methodology and NoC structure of BTS efficiently handle data exchanges for both (i)NTT and the automorphism (see Section 5).

5 BTS MICROARCHITECTURE

We devise a massively parallel architecture that distributes PEs in a grid. A PE consists of functional units (FUs) and an SRAM scratchpad. An NTTU in each PE handles a portion of the residues in a residue polynomial during (i)NTT. By exploiting CLP, the coefficient-wise or element-wise functions can be computed in a PE without any inter-PE data exchange.

Fig. 5 presents a high-level overview of BTS. We arrange 2,048 (n_{PE}) PEs in a grid with a vertical height of 32 (n_{PEver}) and a horizontal width of 64 (n_{PEhor}). The PEs are interconnected via dimension-wise crossbars in the form of 32×32 vertical crossbars (xbar_v) and 64×64 horizontal crossbars (xbar_h). We populate a central, constant memory, storing precomputed values including twiddle factors for (i)NTT and $\hat{q}_j, \hat{q}_j^{-1}$ for BConv. A broadcast unit (BrU) delivers the precomputed values to the PEs at the required moments. Memory controllers are located at the top and bottom sides, each connected to an HBM stack. BTS receives instructions and necessary data from the host via the PCIe interface. The word size in BTS is 64 bits. Modular reduction units use Barrett reduction [9] to bring the 128-bit multiplied results back to the word size.

5.1 Datapath for (i)NTT

BTS maps the coefficients of a polynomial to the PEs suited to 3D-NTT. We view the N residues in a residue polynomial as a $(N_x, N_y, N_z) = (n_{\text{PEhor}}, n_{\text{PEver}}, {}^{N}/_{n_{\text{PE}}})$ cube. Then in the RNS domain, a residue at the coefficient index i (the coefficient of X^i) is at position (x, y, z) in this cube, where $i = x + N_x \cdot y + N_x \cdot N_y \cdot z$. We allocate residues at position $(x', y', z')_{z' \in [0, N_z)}$ of such a cube to the PE of (x', y') coordinate in the PE grid. 3D-NTT is broken down into five steps in BTS. First, we conduct i) NTT_z inside a single PE, which corresponds to the NTT along the z-axis of the cube. Next, ii) data exchanges between vertically aligned PEs are executed, corresponding to n_{PEhor} of yz-plane parallel transposition of residues in the cube. iii) NTT_y along the z-axis follows. iv) Data exchanges between horizontally aligned PEs are executed, corresponding to n_{PEver} of xz-plane parallel transposition of residues in the cube. Finally, v) NTT_x along the z-axis is carried out. iNTT is performed by the reverse process of NTT.

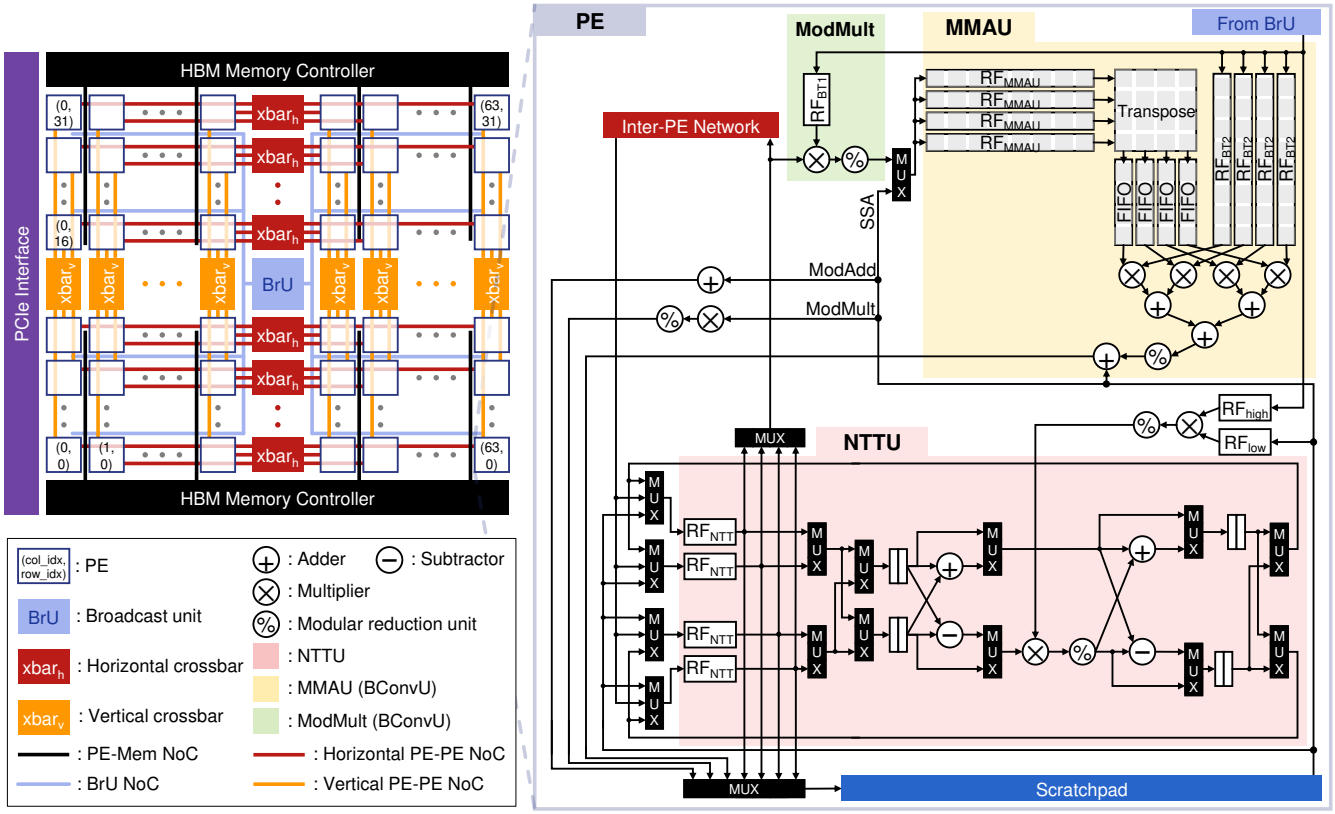


Figure 5: The overview of BTS: Each PE in a grid is denoted as (column index, row index). PEs interconnect through the PE-PE NoC composed of $xbar_v$ and $xbar_h$. BrU is the broadcast unit. BrU and the main memory communicate with PEs through separate NoCs. A PE consists of a scratchpad, an NTTU to undertake NTT/iNTT, a BConvU for BConv, a modular multiplier (ModMult), and a modular adder (ModAdd). BConvU consists of a ModMult and MMAU.

An NTTU supports both NTT and iNTT by using logic circuits similar to [82–85]. We employ separate register files (RF_{NTT}s) to reuse data between (i)NTT stages. An NTTU decomposes NTT_x, NTT_y, and NTT_z into radix-2 NTTs. It is fully pipelined and performs one butterfly op per clock. An input pair is fed in, and an output pair is stored from the NTTU each cycle, provided by two pairs of RF_{NTT}s.

We hide the time for vertical and horizontal data exchanges of 3D-NTT (steps ii) and iv)) through coarse-grained, *epoch*-based pipelining. As steps i), iii), and v) are executed with the same NTTU, we determine the length of an epoch according to the time required to perform these three steps ($\frac{N \log N}{2 \cdot n_{PE}}$ cycles). Within the r-th epoch, we time-multiplex i) of $(r+2)$-th, iii) of r-th, and v) of the $(r-2)$-th residue polynomials, while exchanging ii) of $(r+1)$-th and iv) of the $(r-1)$-th residue polynomials concurrently. Concurrent data exchanges are enabled by separate vertical (ii)) and horizontal (iv)) NoCs. Thus, (i)NTT of a single residue polynomial finishes every epoch.

A single (i)NTT on a residue polynomial requires N different twiddle factors. Because each prime modulus needs different twiddle factors, the sizes of the twiddle factors for (i)NTT on a ciphertext reach dozens of MBs for our target CKKS instances. We reduce the storage for the twiddle factors by decomposing them by means of on-the-fly twiddling (OT) [52]. OT replaces the N-sized precomputed twiddle-factor table with two tables: a *higher-digit table* of ξ_{2N}^{mj} where $1 \le j < {}^{(N-1)}/_{m}$, and a *lower-digit table* of ξ_{2N}^{i} where $1 \le i < m$. We can compose any twiddle factor ξ_{2N}^{k} by multiplying two twiddle factors ξ_{2N}^{i} and ξ_{2N}^{mj} that satisfy $k = mj + i$. OT reduces the memory usage by $2/m$. BTS stores the lower-digit tables of prime moduli in PEs (each PE having different entries) while storing the higher-digit tables in the BrU (all PEs sharing the entries). The BrU broadcasts a higher-digit table for a prime modulus to PEs for every (i)NTT epoch.

5.2 Base Conversion Unit (BConvU)

BConv consists of two parts. The first part multiplies residue polynomials with $\left[\hat{q}_j^{-1}\right]_{q_j}$ and the second part does this with $\left[\hat{q}_j\right]_{p_i}$ and accumulates them. It is the second part that exhibits the coefficient-wise access pattern because it accumulates residues at the same coefficient index in all residue polynomials.

A BConv unit (BConvU) with a modular multiplier (ModMult) for the first part and a modular multiply-accumulate unit (MMAU) for the second part is placed in each PE. BConv strongly depends on the preceding iNTT (see Fig. 3). Because iNTT is a residue-polynomial-wise function, whereas the second part of BConv is

a coefficient-wise function, the MMAU must wait until iNTT is finished on all residue polynomials. We mitigate this by partially overlapping iNTT and BConv. We modify the right-hand side of Eq. 9 as follows:

$$\left\{ \sum_{j_1=0}^{(\ell+1)/l_{\text{sub}}-1} \left[\sum_{j_2=j_1\times l_{\text{sub}}}^{(j_1+1)\times l_{\text{sub}}-1} [[a(X)]_{j_2} \cdot \hat{q}_{j_2}^{-1}]_{q_{j_2}} \cdot \hat{q}_{j_2} \right]_{p_i} \right\}_{0\le i<k} \quad (11)$$

This modification enables the second part to start when the preceding iNTT and the first part of BConv are finished on l_{sub}(= 4 in BTS) residue polynomials and stored in RF_{MMAU}. The MMAU computes the corresponding partial sum (the inner sum of Eq. 11), and accumulates this result with the previous results (the outer sum), which are loaded from and stored on to a scratchpad inducing a read and write every cycle. Temporal registers and FIFO minimize the bandwidth pressure on RF_{MMAU} and transpose the data for the correct orientation to feed l_{sub} lanes into the MMAU. The precomputed values of $[\hat{q_j}^{-1}]_{q_j}$ and $[\hat{q_j}]_{p_i}$ (*BConv tables*) are respectively loaded into the dedicated RF_{BT1} and RF_{BT2} from the BrU when needed.

We also leverage the MMAU for other operations. Subtraction, $1/P$ scaling, and $d0/d1$ addition at the end of key-switching (Fig. 3) can be expressed as $[d2'.ax]_{Q_\ell}\times(1/P) + [d2'.ax]_{P\to Q_\ell}\times(-1/P) + d1\times 1 + 0\times 0$; thus, we fuse these three operations to be computed on the MMAU. We refer to this fusion as subtraction-scaling-addition (SSA).

5.3 Scratchpad

The per-PE scratchpad has three purposes. First, it stores the temporary data generated during the course of the HE ops. The size of the temporal data during key-switching can be large (e.g., a single (i)NTT or BConv can produce 28MB at $\ell+1=28$, $N=2^{17}$). If such data does not reside on-chip, the additional off-chip access would cause severe performance degradation.

Second, the scratchpad also stores the prefetched **evk**. To hide the latency of the **evk** load time, it must be prefetched beforehand. As **evk** is not consumed immediately after being loaded on-chip, it takes up a portion of the scratchpad.

Third, the scratchpad functions as a cache for **c**ts, controlled explicitly by software (SW caching). **c**ts often show high temporal locality during a sequence of HE ops. For instance, during bootstrapping, a **ct** is commonly subjected to multiple HRots. Moreover, as HE ops form a deterministic computational flow and the granularity of cache management is as large as a **ct**, SW control is manageable.

The scratchpad bandwidth demand of the BConvU is high (as later detailed in Fig. 8) due to the accesses involved when updating the partial sums. Considering that the partial sum size is only proportional to k in Eq. 11 and is loaded $(\ell+1)/l_{\text{sub}}$ times, the bandwidth pressure can be relieved by increasing l_{sub}. However, this would also require an increase in the number of lanes in the MMAU (and hence the size of RF_{MMAU}), resulting in a trade-off.

5.4 Network-on-Chip (NoC) design

BTS has three types of on-chip communication: 1) off-chip memory traffic to the PEs (PE-Mem NoC), 2) the distribution of precomputed constants to PEs (BrU NoC), and 3) inter-PE data exchanges for (i)NTT and the automorphism (PE-PE NoC). BTS has a large number of nodes (over 2k endpoints) and requires a high bandwidth. Given the unique communication characteristics of each type of on-chip communication, BTS provides three separate NoCs instead of sharing a single NoC to enable deterministic communication while minimizing the NoC overhead.

PE-Mem NoC: Because data is distributed evenly across the PEs, the off-chip memory (i.e., HBM2e [44]) is placed on the top and bottom and each HBM only needs to communicate with half of the PEs placed nearby. The PE grid placement is exploited by separating the PEs into 32 regions and connecting each HBM pseudo-channel only to a single PE region. An HBM2e stack supports 16 pseudo-channels [62] and thus the upper half of the PEs has 16 regions while the lower half also has 16 regions, with each region consisting of 64 PEs.

BrU NoC: BrU data is globally shared by all PEs and broadcast to all PEs. Given the large number of PEs, the BrU is organized hierarchically with 128 *local BrUs*. Each *local BrU* provides higher-digit tables of twiddle factors and BConv tables to 16 PEs. The global BrU is loaded with all precomputed values before an HE application starts and sends data to the local BrUs that serve as temporary storage/repeaters.

PE-PE NoC: The PE-PE NoC requires support for the highest bandwidth due to the data exchanges necessary between the PEs. The communication pattern is *symmetric* (i.e., each PE sends and receives the same amount of data), and a single PE is not oversubscribed. In addition, because the traffic pattern is known (e.g., all-to-all or a fixed, permutation traffic), the NoC can be greatly simplified. BTS implements a logical 2D flattened butterfly [1, 51] given that communication to other PEs within each row and within each column is limited. However, instead of having a router at each PE, a single "router" xbar_{h} (respectively, xbar_{v}) is shared by all PEs within each row (column); it is placed in the center of each row (column) and used for horizontal (vertical) data exchange steps of (i)NTT (steps ii), iv)). Each xbar_{h} (xbar_{v}) does not require any allocation because the traffic pattern is known ahead of time and can be scheduled through pre-determined arbitration.

5.5 Automorphism

We identify that BTS can handle the automorphism for HRots efficiently. All residues mapped to a single PE always move to another single destination PE under the BTS' PE-coefficient mapping scheme; i.e., the inter-PE communication of the automorphism exhibits a permutation pattern. A PE of the (x', y') PE-grid coordinate holds the residues at positions $(x', y', z')_{z'\in[0,N_z)}$, corresponding to coefficient indices $i = x' + N_x \cdot y' + N_x \cdot N_y \cdot z'$ (Section 5.1). is in binary format only differ in the higher bit-field ($N_x \cdot N_y \cdot z'$), meaning that the automorphism destination indices ($i \cdot 5^r$'s in Eq. 5) also only differ in the higher bit-field; the residues are mapped to the same destination PE corresponding to the lower bit-field ($x'' + N_x \cdot y''$).

We can decompose such a permutation pattern into three steps to fit the PE-PE NoC structure of BTS: intra-PE permutation (z-axis), vertical permutation (y-axis), and horizontal permutation (x-axis). Each step gradually updates the is to $i \cdot 5^r$s from higher to lower bit-fields. The intra-PE permutation process does not use the NoC. The vertical/horizontal permutations can be handled by xbar_{v}/xbar_{h}.

Table 3: The area and the peak power of components in BTS.

Component	Area (μm^2)	Power (mW)	Freq (GHz)
Scratchpad SRAM	114,724	9.86	1.2
RFs	12,479	2.29	Various
NTTU	9,501	12.17	1.2
ModMult (BConvU)	4,070	0.56	0.3
MMAU (BConvU)	9,511	8.42	1.2
Exchange unit	421	1.03	1.2
ModMult	3,833	1.35	0.6
ModAdd	325	0.08	0.6
1 PE	**154,863**	**35.75**	-

Component	Area (mm^2)	Power (W)	Freq (GHz)
2048 PEs	317.2	73.21	-
Inter-PE NoC	3.06	45.93	1.2
Global BrU + NoC	0.42	0.10	0.6
128 local BrUs	3.69	0.04	0.6
HBM2e NoC	0.10	6.81	1.2
2 HBM2e stacks	29.6 [47]	31.76 [66]	-
PCIe5x16 interface	19.6 [47]	5.37 [10]	-
Total	**373.6**	**163.2**	-

The PE-PE NoC can support an arbitrary HRot with any rotation amount (r) without data contention, whose property is similar to that of 3D-NTT.

6 EVALUATION

6.1 Hardware modeling of BTS

We used the ASAP7 [26, 27] design library to synthesize the logic units and datapath components in a 7nm technology node. We simulated the RFs and scratchpads using FinCACTI [76] due to the absence of a public 7nm memory compiler. We updated the analytic models and technology constants of FinCACTI to match ASAP7 and the IRDS roadmap [43]. We validated the RTL synthesis and SRAM simulation results against published information [6, 16, 46, 47, 64, 78, 81].

BTS uses single-ported 128-bit wide 1.2GHz SRAMs for the scratchpads, providing a total capacity of 512MB and a bandwidth of 38.4TB/s chip-wide. RFs are implemented in single-ported SRAMs with variable sizes, port widths, and operating frequencies following the requirements of the FUs. 22MBs of RFs are used chip-wide, providing 292TB/s. Crossbars in the PE-PE NoC have 12-bit wide ports and run at 1.2GHz, providing a bisection bandwidth of 3.6TB/s. The NoC wires are routed over other components [68]. We analyzed the cost of wires and crossbars using FinCACTI and prior works [8, 43, 63, 68]. Two HBM2e stacks are used [44], but with a modest 11% speedup assumed, considering the latest technology [45]. The peak power and area estimation results are shown in Table 3. BTS is 373.6mm^2 in size and consumes up to 163.2W of power.

Table 4: The CKKS instances used for evaluation.

CKKS instance	N	L	dnum	$\log PQ$	λ	Temp data
INS-1	2^{17}	27	1	3090	133.4	183MB
INS-2	2^{17}	39	2	3210	128.7	304MB
INS-3	2^{17}	44	3	3160	130.8	365MB

6.2 Experimental setup

We developed a cycle-level simulator to model the compute capability, latency, and bandwidth of the FUs and the memory components composing BTS. When an HE op is called, the simulator converts the op into a computational graph with primary HE functions. Based on the derived computation and data dependencies, the simulator schedules functions and data loads in epoch granularity while minimizing the temporary data hold time. Utilization rates are also collected and combined with the power model to calculate the energy. The scratchpad space is prioritized in the order of the temporary data, prefetched **evk**, and finally, **ct** caching with an LRU policy.

We measured $T_{mult,a/slot}$ as a microbenchmark and evaluated the most complex applications currently available on CKKS: logistic regression (HELR [39]), CNN inference (ResNet-20 [59]), and sorting [42]. HELR trains a binary classification model with MNIST [34] for 30 iterations, each with a batch containing 1,024 14×14-pixel images. ResNet-20 performs homomorphic convolution, linear transform, and ReLU. It achieves 92.43% accuracy on CIFAR-10 classification [56]. We used the channel packing method proposed in [50] to pack all of the feature map channels into a single **ct** to improve the performance further. Sorting uses a 2-way sorting network to sort 2^{14} data. Because non-linear functions such as ReLU and comparisons are approximated by high-degree polynomial functions in CKKS, they consume many levels and induce hundreds of bootstrapping for ResNet-20 and sorting, respectively.

We compared BTS with the state-of-the-art implementations on a CPU (Lattigo [35]), a GPU (100x [48]), and an ASIC (F1 [75]) for $T_{mult,a/slot}$ and HELR. We ran Lattigo on a system with an Intel Skylake CPU (Xeon Platinum 8160) and 256GB of DDR4-2666 memory. We used the 128b-secure CKKS instance preset of Lattigo and newly implemented HELR on Lattigo. For 100x and F1, the execution times reported in each paper were used. 100x [48] used NVIDIA V100 [65] for the evaluation. We also compared BTS with F1+, whose execution times are optimistically scaled from F1 to have the same area as BTS at 7nm [64]. For other applications, we compared BTS with reported multi-threaded CPU performance from each paper due to the absence of available implementations. We used the CKKS instances shown in Table 4 to evaluate BTS. They all have the same degree and satisfy 128b security but use different values of L and dnum. As dnum and L increase, the temporary data increases, requiring more scratchpad space.

6.3 Performance and efficiency of BTS

Amortized mult time per slot: BTS outperforms the state-of-the-art CPU/GPU/ASIC implementations by tens to thousands of times in terms of the throughput of HMult. Fig. 6 shows the $T_{mult,a/slot}$ values of Lattigo, 100x, F1, F1+ and BTS. The best $T_{mult,a/slot}$ is

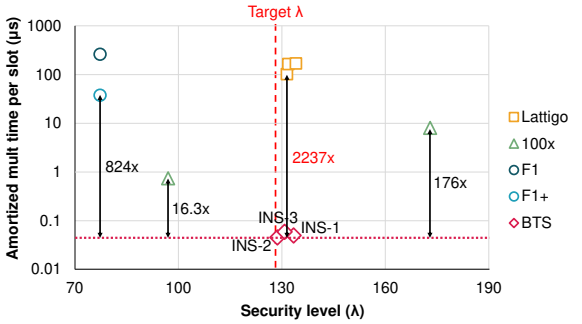


Figure 6: Comparison of the $T_{mult,a/slot}$ between BTS and other prior works of `Lattigo` [35], `100x` [48], and `F1` [75]. `F1+` is a scaled-up version of `F1`. `INS-x` denotes the CKKS instances used for BTS, specified in Table 4.

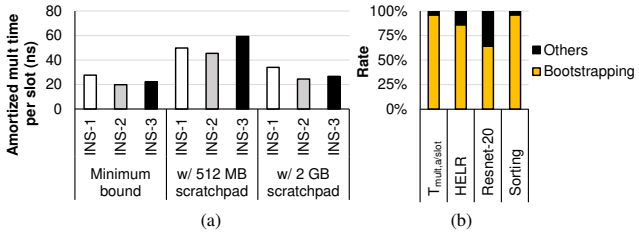


Figure 7: (a) Comparison of the minimum bound of $T_{mult,a/slot}$ (Section 3) and the actual $T_{mult,a/slot}$ using scratchpads of 512MB and 2GB for `INS-x`, and (b) the portion of the bootstrapping time for each application on `INS-1`.

achieved with `INS-2` at 45.5ns, 2,237× better than `Lattigo`. `F1` is even 2.5× slower than `Lattigo`; this occurs because `F1` only supports single-slot bootstrapping.[2] `F1+` is better but shows 824× lower performance than BTS. $T_{mult,a/slot}$ of `100x` is 743ns, reporting the best performance among prior works. However, this is for a 97b-secure parameter set; when using a 173b-secure CKKS instance, `100x` reported a 8μs $T_{mult,a/slot}$.

The performance of `INS-x` is higher than the minimum bound performance shown in Fig. 2 because **ct**s are not always on the scratchpad with limited capacity. Fig. 7(a) shows the minimum and actual $T_{mult,a/slot}$ using 512MB and 2GB of scratchpad for `INS-x`. `INS-2` always performs the best. `INS-1` performs better than `INS-3` with a 512MB scratchpad because the former requires less temporary data, leading to a higher hit rate for **ct**s. With an enough (albeit not practical) scratchpad capacity of 2GB, **ct**s mostly hit, reaching a performance close to the minimum.

Logistic regression: Table 5 reports the average training time per iteration in HELR. Due to the limited parameter set `F1` supports, `F1` only reported the HELR training time for a single iteration with 256 images, which does not require bootstrapping but is not enough for training. We estimated `F1`'s end-to-end HELR performance by assuming that 1024 images in a batch are trained over four iterations,

[2]We call a **ct** sparsely-packed if its corresponding message occupies far fewer slots compared to the maximum number of available ($N/2$). Bootstrapping a sparsely-packed **ct** reduces the computational complexity and consumes fewer levels [17]. In an extreme case using a single-slot, such an effect is maximized. `F1` only supports single-slot bootstrapping due to the lack of multiplicative levels, as it targets support of small parameter sets.

Table 5: Comparison of performance between BTS and other prior works [35, 48, 75] for logistic regression training [39].

	`Lattigo`	`100x`	`F1`	`F1+`	`INS-1`	`INS-2`	`INS-3`
Time (ms)	37,050	775	1,024	148	39.9	28.4	43.5
Speedup	1×	48×	36×	250×	929×	1,306×	852×

Table 6: Evaluating BTS for ResNet-20 inference [59] and sorting [42].

	CPU	`INS-1`	`INS-2`	`INS-3`
ResNet-20 execution time (s)	10,602	1.91	2.02	3.09
Speedup (vs. [59])	1×	5,556×	5,240×	3,427×
# of bootstrapping	-	53	22	19
Sorting execution time (s)	23,066	15.6	18.8	25.2
Speedup (vs. [42])	1×	1,482×	1,226×	915×
# of bootstrapping	-	521	306	229

with $14 \times 14 = 196$ single-slot bootstrapping applied, ignoring the cost of packing/unpacking **ct**s for bootstrapping (giving favor to F1). The execution time with `INS-2` achieves 28.4ms, 1,306×, 27× and 5.2× better than `Lattigo`, `100x` and `F1+`, respectively.

ResNet-20 and sorting: BTS performs up to 5,556× and 1,482× faster over the prior works, [59] and [42] (see Table 6). For ResNet-20, `INS-1` without channel packing shows a 311× speedup. By adopting the channel-packing method [50] exploiting the abundant slots of our target CKKS instances, we reduced the working set and improved the throughput, resulting in an additional 17.8× performance gain and achieving 1.91s of ResNet-20 inference latency on an encrypted image.

Although BTS provides a speedup of more than three orders of magnitude for the most complex applications, these applications still do not fully utilize all 2^{16} slots due to the small problem size. We anticipate the relative speedup of BTS to improve even further when real-world applications are implemented with FHE. For instance, an ImageNet [33] image has over 2^{17} data, which requires multiple fully-packed **ct**s to encrypt.

Parameter selection in retrospect: In Section 3, we estimated the $T_{mult,a/slot}$ of CKKS instances assuming an always-hit scratchpad and used it as a proxy for the performance of FHE applications with frequent bootstrapping. While the $T_{mult,a/slot}$ result from the simulator does not directly match the estimation, the 2GB scratchpad case (Fig 7(a)) does concur. This is because the temporal data of `INS-3` constitutes the largest set (Table 4) and the corresponding hit rate is affected by the scratchpad capacity.

However, $T_{mult,a/slot}$ does not always translate to the application performance for the following reasons. First, when the portion of bootstrapping is relatively small as in ResNet-20 (Fig 7(b)), the complexity of HE ops becomes more influential, and a smaller `dnum` value is better (`INS-1` in Table 6). Second, the better $T_{mult,a/slot}$ caused by deeper levels from higher `dnum`s does not translate to better performance when there exists a level imbalance between **ct**s. Such an imbalance nullifies the benefit of more available levels (see Table 6 with `INS-1` and `INS-2`).

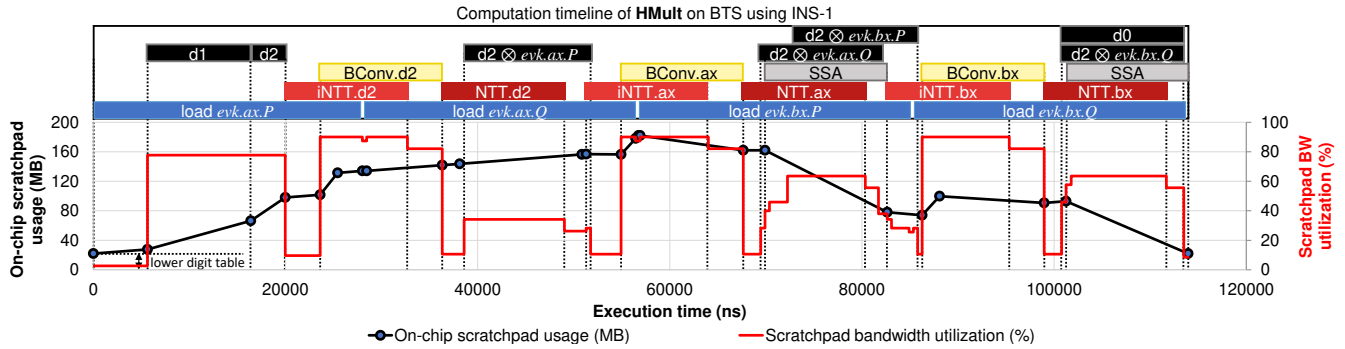


Figure 8: Timeline, on-chip scratchpad usage change, and scratchpad bandwidth utilization change when BTS performs HMult with INS-1.

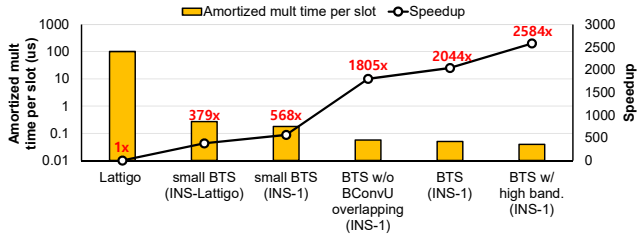


Figure 9: The performance and speedup of $T_{mult,a/slot}$ of BTS when applying various components incrementally. Small BTS is BTS with just enough scratchpad to hold the temporal data of the HE op with no overlapping between BConv and iNTT. The CKKS instance is specified in parentheses.

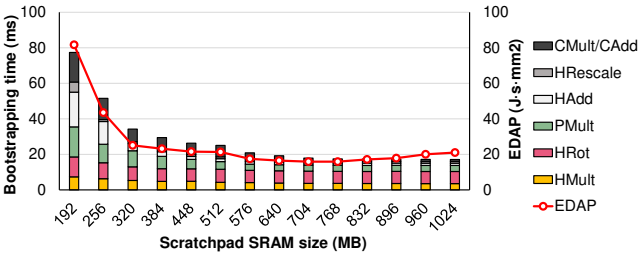


Figure 10: The bootstrapping time and Energy-Delay Area Product (EDAP) of BTS-1 at various scratchpad SRAM sizes.

PE resource utilization over time: Resources populated in PEs are highly utilized while processing HE ops. Fig. 8 presents a detailed timeline of HMult on INS-1 when **ct**s are on the scratchpad. HBM achieves 98% of its peak bandwidth. NTTUs are busy processing (i)NTT of three intermediate polynomials (d2, ax, and bx) 76% of the time. BConv is partially pipelined with iNTT and has strong dependency on the subsequent NTT; thus, it occupies BConvU for 33% of the time. The scratchpad bandwidth requirement of BConv is high because it must load the partial sum for all p_is in Eq. 11 within l_{sub} epochs. BConvU runs SSA while not occupied by BConv.

The bandwidth and capacity utilization of the scratchpad fluctuate over time while being properly provisioned to meet the requirements. The average bandwidth usage was 58.6% over time, peaking at 90% when processing a BConv. The required capacity was also highest at BConv.ax at 183MB.

Ablation study: To evaluate the impact of various attributes of BTS on its performance, first we evaluated a small baseline BTS ($< 230mm^2$) with just enough scratchpad to hold the temporary data that use Lattigo's CKKS instance ($N = 2^{16}$) and without overlapping between BConv and iNTT. The results are 379× faster $T_{mult,a/slot}$ compared to Lattigo. We incrementally changed the CKKS instance to INS-1 and then increased the scratchpad size to 512MB. These changes resulted in 1.50× and 3.18× speedups, respectively (see Fig. 9). Finally, additionally overlapping BConv and iNTT results in a 1.13× speedup, reaching a total of 2044× speedup compared to Lattigo.

We also evaluated BTS with an HBM bandwidth of 2TB/s. We reduced the scratchpad size to make room for the added HBM2e PHYs so that BTS retains the same total area. The result only shows a 1.26× speedup as a larger fraction of time is bound to computations, despite the fact that **evk** load time is halved.

Slowdown of FHE: FHE applications on BTS are still slower than their unencrypted counterparts. HELR is 141× slower and ResNet-20 inference is 440× slower compared to when they are run on a CPU system without FHE. Evaluation of non-polynomial functions such as ReLU, which are costly to evaluate on FHE [57] results in a greater slowdown for ResNet-20. Thus, it is crucial to optimize applications to make them more FHE-friendly.

Impact of the scratchpad size on the performance and EDAP: The performance and energy efficiency of BTS improves as we deploy a larger scratchpad, however becoming saturated as the scratchpad holds most of the HE ops' working sets. Fig. 10 shows the execution time breakdown and energy-delay-area product (EDAP [79]) for the bootstrapping of INS-1 with various scratchpad sizes. We increased the scratchpad size from 192MB (close to the temporary data for HMult) by 64MB, up to 1GB.

With a 192MB scratchpad, BTS frequently load **ct**s from off-chip memory due to capacity misses. At this point, HMult/HRot, which used to be dominant (77% of the bootstrapping time for Lattigo) due to its high computational complexity, now only requires 24% of the execution time. The rest attributes to PMult, HAdd, HRescale, and CMult/CAdd. While BTS greatly reduces the computation time of HMult/HRot with its abundant PEs, the **ct** load time, which any HE ops require when SW cache misses occur, is now dominant.

As the scratchpad size increases, the portion of HMult/HRot on bootstrapping increases. This occurs because the SW cache hit rate

of **ct**s for every HE op gradually increases; 65.6%, 98.8%, 93.7%, 98.6%, 97.5%, and 47.8%, for HMult, HRot, PMult, HAdd, HRescale, and CMult/CAdd, respectively, with a 512MB scratchpad. The execution time of HMult/HRot has a lower-bound of the **evk** load time, even during SW cache hits. However, the other HE ops not requiring **evk** can take significantly less time due to the ratio of the on-chip over the off-chip bandwidth (> 10), when the necessary **ct**s are located on the scratchpad.

7 RELATED WORK

CPU acceleration: [29] parallelized HE ops by multi-threading. [11, 49] leveraged short-SIMD support. [35] exploited the algorithmic analysis from [12] for efficient bootstrapping implementation. Yet other platforms outperform CPU implementations.

GPU acceleration: GPUs are a good fit for accelerating HE ops as they are equipped with a massive number of integer units and abundant memory bandwidth. However, a majority of prior works did not handle bootstrapping [2–4, 49]. [48] was the first work that supported CKKS bootstrapping on GPU. By fusing GPU kernels, [48] reduced off-chip accesses and achieved 242× faster bootstrapping over a CPU. However, the lack of on-chip storage forces some kernels to remain unfused [52]. BTS holds all temporary data on-chip, minimizing off-chip accesses.

FPGA/ASIC acceleration: A different set of works accelerate HE using FPGA or ASIC, but most of them did not consider bootstrapping [53, 54, 71, 73, 74]. HEAX [73] dedicated hardware for CKKS mult on FPGA, reaching a 200× performance gain over a CPU implementation. However, its design is fixed to a limited set of parameters and does not consider bootstrapping. Cheetah [71] introduced algorithmic optimization for an HE-based DNN and proposed an accelerator design suitable for this. Instead of bootstrapping, Cheetah uses multi-party computation (MPC) to mitigate errors during the HE operation. Cheetah sends a ciphertext with error back to the clients and the clients recrypt it as a fresh ciphertext. In MPC, the network latency from the frequent communication with the client limits the performance [80], thus introducing a different challenge compared to FHE. The accelerator design of Cheetah targets a small ciphertext for MPC, which is not suitable for FHE [75]. F1 [75] is the first ASIC design that partially supports bootstrapping. It is a programmable accelerator supporting multiple FHE schemes, including CKKS and BGV. F1 achieves impressive performance on various LHE applications as it provides tailored high-throughput computation units and stores **evk**s on-chip, minimizing the number of off-chip accesses. However, F1 targets the parameter sets with low degree N, thus supporting only non-packed (single-slot) bootstrapping, the throughput of which is greatly exacerbated compared to BTS. F1 is 151.4mm^2 in size at a 12/14nm technology node and shows a TDP of 180.4W excluding the HBM power.

8 CONCLUSION

We have proposed an accelerator architecture for fully homomorphic encryption (FHE), primarily optimized for the throughput of bootstrapping encrypted data. By analyzing the impact of selecting key parameter values on the bootstrapping performance of CKKS, an emerging HE scheme, we devised the design principles of bootstrappable HE accelerators and suggested BTS, which distributes massively-parallel processing elements connected through a network-on-chip design tailored to the unique traffic patterns of number theoretic transform and automorphism, the critical functions of HE operations. We designed BTS to balance off-chip memory accesses, on-chip data reusability, and the computations required for bootstrapping. With BTS, we obtained a speedup of 2,237× in HE multiplication throughput and 5,556× in CNN inference compared to the state-of-the-art CPU implementations.

ACKNOWLEDGMENTS

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2020-0-00840, 40%) and National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. 2020R1A2C2010601, 60%). The EDA tool was supported by the IC Design Education Center (IDEC), Korea. Sangpyo Kim is with the Department of Intelligence and Information, Seoul National University. Jung Ho Ahn, the corresponding author, is with the Department of Intelligence and Information, the Institute of Computer Technology, and the Research Institute for Convergence Science, Seoul National University, Seoul, South Korea.

REFERENCES

[1] Jung Ho Ahn, Nathan L. Binkert, Al Davis, Moray McLaren, and Robert S. Schreiber. 2009. HyperX: Topology, Routing, and Packaging of Efficient Large-Scale Networks. In *SC*. https://doi.org/10.1145/1654059.1654101

[2] Ahmad Al Badawi, Louie Hoang, Chan Fook Mun, Kim Laine, and Khin Mi Mi Aung. 2020. Privft: Private and Fast Text Classification with Homomorphic Encryption. *IEEE Access* 8 (2020), 226544–226556. https://doi.org/10.1109/ACCESS.2020.3045465

[3] Ahmad Al Badawi, Yuriy Polyakov, Khin Mi Mi Aung, Bharadwaj Veeravalli, and Kurt Rohloff. 2019. Implementation and Performance Evaluation of RNS Variants of the BFV Homomorphic Encryption Scheme. *IEEE Transactions on Emerging Topics in Computing* 9, 2 (2019), 941–956. https://doi.org/10.1109/TETC.2019.2902799

[4] Ahmad Al Badawi, Bharadwaj Veeravalli, Chan Fook Mun, and Khin Mi Mi Aung. 2018. High-Performance FV Somewhat Homomorphic Encryption on GPUs: An Implementation Using CUDA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018, 2 (2018), 143–163. https://doi.org/10.13154/tches.v2018.i2.70-95

[5] Martin R. Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin E. Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. 2019. Homomorphic Encryption Standard. *IACR Cryptology ePrint Archive* 939 (2019).

[6] Chris Auth, A. Aliyarukunju, M. Asoro, D. Bergstrom, V. Bhagwat, J. Birdsall, N. Bisnik, M. Buehler, V. Chikarmane, G. Ding, Q. Fu, H. Gomez, W. Han, D. Hanken, M. Haran, M. Hattendorf, R. Heussner, H. Hiramatsu, B. Ho, S. Jaloviar, I. Jin, S. Joshi, S. Kirby, S. Kosaraju, H. Kothari, G. Leatherman, K. Lee, J. Leib, A. Madahavan, K. Marla, H. Meyer, T. Mule, C. Parker, S. Parthasarathy, C. Pelto, L. Pipes, I. Post, M. Prince, A. Rahman, S. Rajamani, A. Saha, J. Dacuna Santos, M. Sharma, V. Sharma, J. Shin, P. Sinha, P. Smith, M. Sprinkle, A. St. Amour, C. Staus, R. Suri, D. Towner, A. Tripathi, A. Tura, C. Ward, and A. Yeoh. 2017. A 10nm High Performance and Low-Power CMOS Technology Featuring 3rd Generation FinFET Transistors, Self-Aligned Quad Patterning, Contact over Active Gate and Cobalt Local Interconnects. In *IEEE International Electron Devices Meeting*. https://doi.org/10.1109/IEDM.2017.8268472

[7] Jean-Claude Bajard, Julien Eynard, M. Anwar Hasan, and Vincent Zucca. 2016. A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes. In *Selected Areas in Cryptography*. https://doi.org/10.1007/978-3-319-69453-5_23

[8] Kaustav Banerjee and Amit Mehrotra. 2002. A Power-Optimal Repeater Insertion Methodology for Global Interconnects in Nanometer Designs. *IEEE Transactions on Electron Devices* 49, 11 (2002), 2001–2007. https://doi.org/10.1109/TED.2002.804706

[9] Paul Barrett. 1986. Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In *Annual International Conference on the Theory and Application of Cryptographic Techniques*. https://doi.org/10.5555/36664.36688

[10] Mike Bichan, Clifford Ting, Bahram Zand, Jing Wang, Ruslana Shulyzki, James Guthrie, Katya Tyshchenko, Junhong Zhao, Alireza Parsafar, Eric Liu, Aynaz Vatankhahghadim, Shaham Sharifian, Aleksey Tyshchenko, Michael De Vita, Syed Rubab, Sitaraman Iyer, Fulvio Spagna, and Noam Dolev. 2020. A 32Gb/s NRZ 37dB SerDes in 10nm CMOS to Support PCI Express Gen 5 Protocol. In *IEEE Custom Integrated Circuits Conference*. https://doi.org/10.1109/CICC48029.2020.9075947

[11] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe D. M. de Souza, and Vinodh Gopal. 2021. Intel HEXL: Accelerating Homomorphic Encryption with Intel AVX512-IFMA52. In *Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. https://doi.org/10.1145/3474366.3486926

[12] Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. https://doi.org/10.1007/978-3-030-77870-5_21

[13] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Transactions on Computing Theory* 6, 3 (2014). https://doi.org/10.1145/2633600

[14] Zvika Brakerski and Vinod Vaikuntanathan. 2014. Efficient Fully Homomorphic Encryption from (Standard) LWE. *SIAM J. Comput.* 43, 2 (2014), 831–871. https://doi.org/10.1137/120868669

[15] Alon Brutzkus, Ran Gilad-Bachrach, and Oren Elisha. 2019. Low Latency Privacy Preserving Inference. In *International Conference on Machine Learning*, Vol. 97. 812–821.

[16] Jonathan Chang, Yen-Huei Chen, Wei-Min Chan, Sahil Preet Singh, Hank Cheng, Hidehiro Fujiwara, Jih-Yu Lin, Kao-Cheng Lin, John Hung, Robin Lee, Hung-Jen Liao, Jhon-Jhy Liaw, Quincy Li, Chih-Yung Lin, Mu-Chi Chiang, and Shien-Yang Wu. 2017. 12.1 A 7nm 256Mb SRAM in High-K Metal-Gate FinFET Technology with Write-Assist Circuitry for Low-VMIN Applications. In *IEEE International Solid-State Circuits Conference*. https://doi.org/10.1109/ISSCC.2017.7870333

[17] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Improved Bootstrapping for Approximate Homomorphic Encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. https://doi.org/10.1007/978-3-030-17656-3_2

[18] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *ISCA*. https://doi.org/10.1109/ISCA.2016.40

[19] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. A Full RNS Variant of Approximate Homomorphic Encryption. In *Selected Areas in Cryptography*. https://doi.org/10.1007/978-3-030-10970-7_16

[20] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. https://doi.org/10.1007/978-3-319-78381-9_14

[21] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. 2019. A Hybrid of Dual and Meet-in-the-Middle Attack on Sparse and Rernary Secret LWE. *IEEE Access* 7 (2019), 89497–89506. https://doi.org/10.1109/ACCESS.2019.2925425

[22] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *International Conference on the Theory and Applications of Cryptology and Information Security*. https://doi.org/10.1007/978-3-319-70694-8_15

[23] Jung Hee Cheon, Yongha Son, and Donggeon Yhee. 2022. Practical FHE Parameters against Lattice Attacks. *Journal of the Korean Mathematical Society* 59, 1 (2022), 35–51. https://doi.org/10.4134/JKMS.j200650

[24] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33, 1 (2020), 34–91. https://doi.org/10.1007/s00145-019-09319-x

[25] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. NVIDIA A100 Tensor Core GPU: Performance and Innovation. *IEEE Micro* 41, 2 (2021), 29–35. https://doi.org/10.1109/MM.2021.3061394

[26] Lawrence T Clark, Vinay Vashishtha, David M Harris, Samuel Dietrich, and Zunyan Wang. 2017. Design Flows and Collateral for the ASAP7 7nm FinFET Predictive Process Design Kit. In *IEEE International Conference on Microelectronic Systems Education*. https://doi.org/10.1109/MSE.2017.7945071

[27] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm FinFET Predictive Process Design Kit. *Microelectronics Journal* 53 (2016), 105–115. https://doi.org/10.1016/j.mejo.2016.04.006

[28] James W. Cooley and John W. Tukey. 1965. An Algorithm for the Machine Calculation of Complex Fourier Series. *Math. Comp.* 19, 90 (1965), 297–301. https://doi.org/10.1090/s0025-5718-1965-0178586-1

[29] CryptoLab Inc. 2018. HEAAN v2.1. https://github.com/snucrypto/HEAAN

[30] Benjamin R. Curtis and Rachel Player. 2019. On the Feasibility and Impact of Standardising Sparse-secret LWE Parameter Sets for Homomorphic Encryption. In *ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. https://doi.org/10.1145/3338469.3358940

[31] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. 2012. Multiparty Computation from Somewhat Homomorphic Encryption. In *Annual International Cryptology Conference*. https://doi.org/10.1007/978-3-642-32009-5_38

[32] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An Encrypted Vector Arithmetic Language and Compiler for Efficient Homomorphic Computation. In *ACM SIGPLAN International Conference on Programming Language Design and Implementation*. https://doi.org/10.1145/3385412.3386023

[33] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition*. https://doi.org/10.1109/CVPR.2009.5206848

[34] Li Deng. 2012. The MNIST Database of Handwritten Digit Images for Machine Learning Research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142. https://doi.org/10.1109/MSP.2012.2211477

[35] EPFL-LDS. 2021. Lattigo v2.3.0. https://github.com/ldsec/lattigo

[36] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *IACR Cryptology ePrint Archive* 144 (2012).

[37] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *ACM Symposium on Theory of Computing*. https://doi.org/10.1145/1536414.1536440

[38] Craig Gentry and Shai Halevi. 2011. Implementing Gentry's Fully-Homomorphic Encryption Scheme. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. https://doi.org/10.1007/978-3-642-20465-4_9

[39] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2019. Logistic Regression on Homomorphic Encrypted Data at Scale. In *AAAI Conference on Artificial Intelligence*. https://doi.org/10.1609/aaai.v33i01.33019466

[40] Kyoohyung Han and Dohyeong Ki. 2020. Better Bootstrapping for Approximate Homomorphic Encryption. In *Cryptographers' Track at the RSA Conference*. https://doi.org/10.1007/978-3-030-40186-3_16

[41] Ron Ho, Kenneth Mai, and Mark Horowitz. 2001. The Future of Wires. *Proc. IEEE* 89, 4 (2001), 490–504. https://doi.org/10.1109/5.920580

[42] Seungwan Hong, Seunghong Kim, Jiheon Choi, Younho Lee, and Jung Hee Cheon. 2021. Efficient Sorting of Homomorphic Encrypted Data With k-Way Sorting Network. *IEEE Transactions on Information Forensics and Security* 16 (2021), 4389–4404. https://doi.org/10.1109/TIFS.2021.3106167

[43] IEEE. 2018. *International Roadmap for Devices and Systems: 2018.* Technical Report. https://irds.ieee.org/editions/2018/

[44] JEDEC. 2021. *High Bandwidth Memory (HBM) DRAM.* Technical Report JESD235D.

[45] JEDEC. 2022. *High Bandwidth Memory DRAM (HBM3).* Technical Report JESD238.

[46] W.C. Jeong, S. Maeda, H.J. Lee, K.W. Lee, T.J. Lee, D.W. Park, B.S. Kim, J.H. Do, T. Fukai, D.J. Kwon, K.J. Nam, W.J. Rim, M.S. Jang, H.T. Kim, Y.W. Lee, J.S. Park, E.C. Lee, D.W. Ha, C.H. Park, H.J. Cho, S.M. Jung, and H.K. Kang. 2018. True 7nm Platform Technology featuring Smallest FinFET and Smallest SRAM cell by EUV, Special Constructs and 3rd Generation Single Diffusion Break. In *IEEE Symposium on VLSI Technology*. https://doi.org/10.1109/VLSIT.2018.8510682

[47] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter C. Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David A. Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4i: Industrial Product. In *ISCA*. https://doi.org/10.1109/ISCA52012.2021.00010

[48] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 4 (2021), 114−−148. https://doi.org/10.46586/tches.v2021.i4.114-148

[49] Wonkyung Jung, Eojin Lee, Sangpyo Kim, Jongmin Kim, Namhoon Kim, Keewoo Lee, Chohong Min, Jung Hee Cheon, and Jung Ho Ahn. 2021. Accelerating Fully Homomorphic Encryption Through Architecture-Centric Analysis and Optimization. *IEEE Access* 9 (2021), 98772–98789. https://doi.org/10.1109/ACCESS.2021.3096189

[50] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. 2018. {GAZELLE}: A Low Latency Framework for Secure Neural Network Inference. In *USENIX Security Symposium*.

[51] John Kim, James Balfour, and William Dally. 2007. Flattened Butterfly Topology for On-Chip Networks. In *MICRO*. 172–182. https://doi.org/10.1109/MICRO.2007.29

[52] Sangpyo Kim, Wonkyung Jung, Jaiyoung Park, and Jung Ho Ahn. 2020. Accelerating Number Theoretic Transformations for Bootstrappable Homomorphic Encryption on GPUs. In *IEEE International Symposium on Workload Characterization*. https://doi.org/10.1109/IISWC50251.2020.00033

[53] Sunwoong Kim, Keewoo Lee, Wonhee Cho, Jung Hee Cheon, and Rob A. Rutenbar. 2019. FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption. In *International Conference on ReConFigurable Computing and FPGAs*. https://doi.org/10.1109/ReConFig48160.2019.8994793

[54] Sunwoong Kim, Keewoo Lee, Wonhee Cho, Yujin Nam, Jung Hee Cheon, and Rob A. Rutenbar. 2020. Hardware Architecture of a Number Theoretic Transform for a Bootstrappable RNS-based Homomorphic Encryption Scheme. In *IEEE International Symposium on Field-Programmable Custom Computing Machines*. https://doi.org/10.1109/FCCM48280.2020.00017

[55] Simon Knowles. 2021. Graphcore. In *IEEE Hot Chips 33 Symposium*. https://doi.org/10.1109/HCS52781.2021.9567075

[56] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.

[57] Junghyun Lee, Eunsang Lee, Joon-Woo Lee, Yongjune Kim, Young-Sik Kim, and Jong-Seon No. 2021. Precise Approximation of Convolutional Neural Networks for Homomorphically Encrypted Data. *arXiv preprint arXiv:2105.10879* (2021).

[58] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. 2021. High-Precision Bootstrapping of RNS-CKKS Homomorphic Encryption Using Optimal Minimax Polynomial Approximation and Inverse Sine Function. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. https://doi.org/10.1007/978-3-030-77870-5_22

[59] Joon-Woo Lee, Hyungchul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, and Jong-Seon No. 2022. Privacy-Preserving Machine Learning With Fully Homomorphic Encryption for Deep Neural Network. *IEEE Access* 10 (2022), 30039–30054. https://doi.org/10.1109/ACCESS.2022.3159694

[60] Yongwoo Lee, Joonwoo Lee, Young-Sik Kim, HyungChul Kang, and Jong-Seon No. 2020. High-Precision and Low-Complexity Approximate Homomorphic Encryption by Error Variance Minimization. *IACR Cryptology ePrint Archive* 1549 (2020).

[61] Eitan Medina and Eran Dagan. 2020. Habana Labs Purpose-Built AI Inference and Training Processor Architectures: Scaling AI Training Systems Using Standard Ethernet With Gaudi Processor. *IEEE Micro* 40, 2 (2020), 17–24. https://doi.org/10.1109/MM.2020.2975185

[62] Micron Technology, Inc. 2020. *8GB/16GB HBM2E with ECC*. Technical Report CCM005-1412786195-10301 - Rev. D 08/2020 EN. https://media-www.micron.com/-/media/client/global/documents/products/data-sheet/dram/hbm2e/8gb_and_16gb_hbm2e_dram.pdf?rev=dbfcf653271041a497e5f1bef1a169ca

[63] Peter Moon, Vinay Chikarmane, Kevin Fischer, Rohit Grover, Tarek A Ibrahim, Doug Ingerly, Kevin J Lee, Chris Litteken, Tony Mule, and Sarah Williams. 2008. Process and Electrical Results for the On-die Interconnect Stack for Intel's 45nm Process Generation. *Intel Technology Journal* 12, 2 (2008).

[64] S. Narasimha, B. Jagannathan, A. Ogino, D. Jaeger, B. Greene, C. Sheraw, K. Zhao, B. Haran, U. Kwon, A. K. M. Mahalingam, B. Kannan, B. Morganfeld, J. Dechene, C. Radens, A. Tessier, A. Hassan, H. Narisetty, I. Ahsan, M. Aminpur, C. An, M. Aquilino, A. Arya, R. Augur, N. Baliga, R. Bhelkar, G. Biery, A. Blauberg, N. Borjemscaia, A. Bryant, L. Cao, V. Chauhan, M. Chen, L. Cheng, J. Choo, C. Christiansen, T. Chu, B. Cohen, R. Coleman, D. Conklin, S. Crown, A. da Silva, D. Dechene, G. Derderian, S. Deshpande, G. Dilliway, K. Donegan, M. Eller, Y. Fan, Q. Fang, A. Gassaria, R. Gauthier, S. Ghosh, G. Gifford, T. Gordon, M. Gribelyuk, G. Han, J.H. Han, K. Han, M. Hasan, J. Higman, J. Holt, L. Hu, L. Huang, C. Huang, T. Hung, Y. Jin, J. Johnson, S. Johnson, V. Joshi, M. Joshi, P. Justison, S. Kalaga, T. Kim, W. Kim, R. Krishnan, B. Krishnan, K. Anil, M. Kumar, J. Lee, R. Lee, J. Lemon, S.L. Liew, P. Lindo, M. Lingalugari, M. Lipinski, P. Liu, J. Liu, S. Lucarini, W. Ma, E. Maciejewski, S. Madisetti, A. Malinowski, J. Mehta, C. Meng, S. Mitra, C. Montgomery, H. Nayfeh, T. Nigam, G. Northrop, K. Onishi, C. Ordonio, M. Ozbek, R. Pal, S. Parihar, O. Patterson, E. Ramanathan, I. Ramirez, R. Ranjan, J. Sarad, V. Sardesai, S. Saudari, C. Schiller, B. Senapati, C. Serrau, N. Shah, T. Shen, H. Sheng, J. Shepard, Y. Shi, M.C. Silvestre, D. Singh, Z. Song, J. Sporre, P. Srinivasan, Z. Sun, A. Sutton, R. Sweeney, K. Tabakman, M. Tan, X. Wang, E. Woodard, G. Xu, D. Xu, T. Xuan, Y. Yan, J. Yang, K.B. Yeap, M. Yu, A. Zainuddin, J. Zeng, K. Zhang, M. Zhao, Y. Zhong, R. Carter, C.H. Lin, S. Grunow, C. Child, M. Lagus, R. Fox, E. Kaste, G. Gomba, S. Samavedam, P. Agnello, and D. K. Sohn. 2017. A 7nm CMOS Technology Platform for Mobile and High Performance Compute Application. In *IEEE International Electron Devices Meeting*. https://doi.org/10.1109/IEDM.2017.8268476

[65] NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture*. Technical Report WP-08608-001_v1.1. https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[66] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W Keckler, and William J Dally. 2017. Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems. In *MICRO*. https://doi.org/10.1145/3123939.3124545

[67] PALISADE Project. 2021. PALISADE Lattice Cryptography Library (release 1.11.5). https://palisade-crypto.org/

[68] Giorgos Passas, Manolis Katevenis, and Dionisios Pnevmatikatos. 2012. Crossbar NoCs are Scalable Beyond 100 Nodes. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31, 4 (2012), 573–585. https://doi.org/10.1109/TCAD.2011.2176730

[69] Raghu Prabhakar and Sumti Jairath. 2021. SambaNova SN10 RDU: Accelerating Software 2.0 with Dataflow. In *IEEE Hot Chips 33 Symposium*. https://doi.org/10.1109/HCS52781.2021.9567250

[70] Parthasarathy Ranganathan, Daniel Stodolsky, Jeff Calow, Jeremy Dorfman, Marisabel Guevara, Clinton Wills Smullen IV, Aki Kuusela, Raghu Balasubramanian, Sandeep Bhatia, Prakash Chauhan, Anna Cheung, In Suk Chong, Niranjani Dasharathi, Jia Feng, Brian Fosco, Samuel Foss, Ben Gelb, Sara J. Gwin, Yoshiaki Hase, Da-ke He, C. Richard Ho, Roy W. Huffman Jr., Elisha Indupalli, Indira Jayaram, Poonacha Kongetira, Cho Mon Kyaw, Aaron Laursen, Yuan Li, Fong Lou, Kyle A. Lucke, JP Maaninen, Ramon Macias, Maire Mahony, David Alexander Munday, Srikanth Muroor, Narayana Penukonda, Eric Perkins-Argueta, Devin Persaud, Alex Ramirez, Ville-Mikko Rautio, Yolanda Ripley, Amir Salek, Sathish Sekar, Sergey N. Sokolov, Rob Springer, Don Stark, Mercedes Tan, Mark S. Wachsler, Andrew C. Walton, David A. Wickeraad, Alvin Wijaya, and Hon Kwan Wu. 2021. Warehouse-Scale Video Acceleration: Co-Design and Deployment in the Wild. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. https://doi.org/10.1145/3445814.3446723

[71] Brandon Reagen, Woo-Seok Choi, Yeongil Ko, Vincent T. Lee, Hsien-Hsin S. Lee, Gu-Yeon Wei, and David Brooks. 2021. Cheetah: Optimizing and Accelerating Homomorphic Encryption for Private Inference. In *HPCA*. https://doi.org/10.1109/HPCA51647.2021.00013

[72] Oded Regev. 2009. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *J. ACM* 56, 6 (2009), 40 pages. https://doi.org/10.1145/1568318.1568324

[73] M. Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An Architecture for Computing on Encrypted Data. In *ASPLOS*. https://doi.org/10.1145/3373376.3378523

[74] Sujoy Sinha Roy, Furkan Turan, Kimmo Järvinen, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. FPGA-Based High-Performance Parallel Architecture for Homomorphic Computing on Encrypted Data. In *HPCA*. https://doi.org/10.1109/HPCA.2019.00052

[75] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In *MICRO*. https://doi.org/10.1145/3466752.3480070

[76] Alireza Shafaei, Yanzhi Wang, Xue Lin, and Massoud Pedram. 2014. FinCACTI: Architectural Analysis and Modeling of Caches with Deeply-Scaled FinFET Devices. In *IEEE Computer Society Annual Symposium on VLSI*. https://doi.org/10.1109/ISVLSI.2014.94

[77] Yongha Son. 2021. SparseLWE-estimator. https://github.com/Yongyongha/SparseLWE-estimator

[78] Taejoong Song, Jonghoon Jung, Woojin Rim, Hoonki Kim, Yongho Kim, Changnam Park, Jeongho Do, Sunghyun Park, Sungwee Cho, Hyuntaek Jung, Bongjae Kwon, Hyun-Su Choi, Jaeseung Choi, and Jong Shik Yoon. 2018. A 7nm FinFET SRAM Using EUV Lithography with Dual Write-Driver-Assist Circuitry for Low-Voltage Applications. In *IEEE International Solid-State Circuits Conference*. https://doi.org/10.1109/ISSCC.2018.8310252

[79] Shyamkumar Thoziyoor, Jung Ho Ahn, Matteo Monchiero, Jay B. Brockman, and Norman P. Jouppi. 2008. A Comprehensive Memory Modeling Tool and Its Application to the Design and Analysis of Future Memory Hierarchies. In *ISCA*. https://doi.org/10.1145/1394608.1382127

[80] McKenzie van der Hagen and Brandon Lucia. 2021. Practical encrypted computing for iot clients. *arXiv preprint arXiv:2103.06743* (2021).

[81] Shien-Yang Wu, C.Y. Lin, M.C. Chiang, J.J. Liaw, J.Y. Cheng, S.H. Yang, C.H. Tsai, P.N. Chen, T. Miyashita, C.H. Chang, V.S. Chang, K.H. Pan, J.H. Chen, Y.S. Mor, K.T. Lai, C.S. Liang, H.F. Chen, S.Y. Chang, C.J. Lin, C.H. Hsieh, R.F. Tsui, C.H. Yao, C.C. Chen, R. Chen, C.H. Lee, H.J. Lin, C.W. Chang, K.W. Chen, M.H. Tsai, K.S. Chen, Y. Ku, and S.M. Jang. 2016. A 7nm CMOS Platform Technology Featuring 4th Generation FinFET Transistors with a 0.027um2 High Density 6-T SRAM cell for Mobile SoC Applications. In *IEEE International Electron Devices Meeting*. https://doi.org/10.1109/IEDM.2016.7838333

[82] Guozhu Xin, Jun Han, Tianyu Yin, Yuchao Zhou, Jianwei Yang, Xu Cheng, and Xiaoyang Zeng. 2020. VPQC: A Domain-Specific Vector Processor for Post-Quantum Cryptography Based on RISC-V Architecture. *IEEE Transactions on Circuits and Systems I: Regular Papers* 67, 8 (2020), 2672–2684. https://doi.org/10.1109/TCSI.2020.2983185

[83] Guozhu Xin, Yifan Zhao, and Jun Han. 2021. A Multi-Layer Parallel Hardware Architecture for Homomorphic Computation in Machine Learning. In *IEEE International Symposium on Circuits and Systems*. https://doi.org/10.1109/ISCAS51556.2021.9401623

[84] Yufei Xing and Shuguo Li. 2021. A Compact Hardware Implementation of CCA-secure Key Exchange Mechanism CRYSTALS-KYBER on FPGA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 2 (2021), 328–356. https://doi.org/10.46586/tches.v2021.i2.328-356

[85] Ye Zhang, Shuo Wang, Xian Zhang, Jiangbin Dong, Xingzhong Mao, Fan Long, Cong Wang, Dong Zhou, Mingyu Gao, and Guangyu Sun. 2021. PipeZK: Accelerating Zero-Knowledge Proof with a Pipelined Architecture. In *ISCA*. https://doi.org/10.1109/ISCA52012.2021.00040